CHAPTER 19

Integral Equations and Inverse Theory

19.0 Introduction

Many people, otherwise numerically knowledgable, imagine that the numerical solution of integral equations must be an extremely arcane topic, since, until recently, it was almost never treated in numerical analysis textbooks. Actually there is a large and growing literature on the numerical solution of integral equations, including several good monographs [1-3]. One reason for the sheer volume of this activity is that there are many different kinds of equations, each with many different possible pitfalls; often many different algorithms have been proposed to deal with a single case.

There is a close correspondence between linear integral equations, which specify linear, integral relations among functions in an infinite-dimensional function space, and plain old linear equations, which specify analogous relations among vectors in a finite-dimensional vector space. Because this correspondence lies at the heart of most computational algorithms, it is worth making it explicit as we recall how integral equations are classified.

Fredholm equations involve definite integrals with fixed upper and lower limits. An *inhomogeneous Fredholm equation of the first kind* has the form

$$g(t) = \int_a^b K(t,s)\, f(s)\, ds \tag{19.0.1}$$

Here $f(t)$ is the unknown function to be solved for, while $g(t)$ is a known "right-hand side." (In integral equations, for some odd reason, the familiar "right-hand side" is conventionally written on the left!) The function of two variables, $K(t,s)$, is called the *kernel*. Equation (19.0.1) is analogous to the matrix equation

$$\mathbf{K} \cdot \mathbf{f} = \mathbf{g} \tag{19.0.2}$$

whose solution is $\mathbf{f} = \mathbf{K}^{-1} \cdot \mathbf{g}$, where $\mathbf{K}^{-1}$ is the matrix inverse. Like equation (19.0.2), equation (19.0.1) has a unique solution whenever g is nonzero (the homogeneous case with $g = 0$ is almost never useful) and K is invertible. However, as we shall see, this latter condition is as often the exception as the rule.

The analog of the finite-dimensional eigenvalue problem

$$(\mathbf{K} - \sigma \mathbf{1}) \cdot \mathbf{f} = \mathbf{g} \tag{19.0.3}$$

is called a *Fredholm equation of the second kind*, usually written

$$f(t) = \lambda \int_a^b K(t,s) f(s)\, ds + g(t) \tag{19.0.4}$$

Again, the notational conventions do not exactly correspond: λ in equation (19.0.4) is $1/\sigma$ in (19.0.3), while $\mathbf{g}$ is $-g/\lambda$. If g (or $\mathbf{g}$) is zero, then the equation is said to be *homogeneous*. If the kernel $K(t,s)$ is bounded, then, like equation (19.0.3), equation (19.0.4) has the property that its homogeneous form has solutions for at most a denumerably infinite set $\lambda = \lambda_n$, $n = 1, 2, \ldots$, the *eigenvalues*. The corresponding solutions $f_n(t)$ are the *eigenfunctions*. The eigenvalues are real if the kernel is symmetric.

In the *inhomogeneous* case of nonzero g (or $\mathbf{g}$), equations (19.0.3) and (19.0.4) are soluble *except* when λ (or σ) is an eigenvalue — because the integral operator (or matrix) is singular then. In integral equations this dichotomy is called *the Fredholm alternative*.

Fredholm equations of the first kind are often extremely ill-conditioned. Applying the kernel to a function is generally a smoothing operation, so the solution, which requires inverting the operator, will be extremely sensitive to small changes or errors in the input. Smoothing often actually loses information, and there is no way to get it back in an inverse operation. Specialized methods have been developed for such equations, which are often called *inverse problems*. In general, a method must augment the information given with some prior knowledge of the nature of the solution. This prior knowledge is then used, in one way or another, to restore lost information. We will introduce such techniques in §19.4.

Inhomogeneous Fredholm equations of the second kind are much less often ill-conditioned. Equation (19.0.4) can be rewritten as

$$\int_a^b [K(t,s) - \sigma \delta(t-s)] f(s)\, ds = -\sigma g(t) \tag{19.0.5}$$

where $\delta(t-s)$ is a Dirac delta function (and where we have changed from λ to its reciprocal σ for clarity). If σ is large enough in magnitude, then equation (19.0.5) is, in effect, diagonally dominant and thus well-conditioned. Only if σ is small do we go back to the ill-conditioned case.

Homogeneous Fredholm equations of the second kind are likewise not particularly ill-posed. If K is a smoothing operator, then it will map many f's to zero, or near-zero; there will thus be a large number of degenerate or nearly degenerate eigenvalues around $\sigma = 0$ ($\lambda \to \infty$), but this will cause no particular computational difficulties. In fact, we can now see that the magnitude of σ needed to rescue the inhomogeneous equation (19.0.5) from an ill-conditioned fate is generally much *less* than that required for diagonal dominance. Since the σ term shifts all eigenvalues, it is enough that it be large enough to shift a smoothing operator's forest of near-zero eigenvalues away from zero, so that the resulting operator becomes invertible (except, of course, at the discrete eigenvalues).

Volterra equations are a special case of Fredholm equations with $K(t,s) = 0$ for $s > t$. Chopping off the unnecessary part of the integration, Volterra equations are written in a form where the upper limit of integration is the independent variable t. The *Volterra equation of the first kind*,

$$g(t) = \int_a^t K(t,s)\, f(s)\, ds \tag{19.0.6}$$

has as its analog the matrix equation (now written out in components)

$$\sum_{j=0}^{k} K_{kj}\, f_j = g_k \tag{19.0.7}$$

Comparing with equation (19.0.2), we see that the Volterra equation corresponds to a matrix $\mathbf{K}$ that is lower (i.e., left) triangular, with zero entries above the diagonal. As we know from Chapter 2, such matrix equations are trivially soluble by forward substitution. Techniques for solving Volterra equations are similarly straightforward. When experimental measurement noise does not dominate, Volterra equations of the first kind tend *not* to be ill-conditioned; the upper limit to the integral introduces a sharp step that conveniently spoils any smoothing properties of the kernel.

The Volterra equation of the second kind is written

$$f(t) = \int_a^t K(t,s)\, f(s)\, ds + g(t) \tag{19.0.8}$$

whose matrix analog is the equation

$$(\mathbf{K} - \mathbf{1}) \cdot \mathbf{f} = \mathbf{g} \tag{19.0.9}$$

with $\mathbf{K}$ lower triangular. The reason there is no λ in these equations is that (i) in the inhomogeneous case (nonzero g) it can be absorbed into K, while (ii) in the homogeneous case ($g = 0$), it is a theorem that Volterra equations of the second kind with bounded kernels have no eigenvalues with square-integrable eigenfunctions.

We have specialized our definitions to the case of linear integral equations. In a nonlinear version of equation (19.0.1) or (19.0.6), instead of $K(t,s)\, f(s)$ the integrand would be $K(t, s, f(s))$. A nonlinear version of equation (19.0.4) or (19.0.8) would have an integrand $K(t, s, f(t), f(s))$. Nonlinear Fredholm equations are considerably more complicated than their linear counterparts. Fortunately, they do not occur as frequently in practice and we shall by and large ignore them in this chapter. By contrast, solving nonlinear Volterra equations usually involves only a slight modification of the algorithm for linear equations, as we shall see.

Almost all methods for solving integral equations numerically make use of *quadrature rules*, frequently Gaussian quadratures. This would be a good time for you to go back and review §4.6, especially the advanced material toward the end of that section.

In the sections that follow, we first discuss Fredholm equations of the second kind with smooth kernels (§19.1). Nontrivial quadrature rules come into the discussion, but we will be dealing with well-conditioned systems of equations. We then return to Volterra equations (§19.2), and find that simple and straightforward methods are generally satisfactory for these equations.

In §19.3 we discuss how to proceed in the case of singular kernels, focusing largely on Fredholm equations (both first and second kinds). Singularities require special quadrature rules, but they are also sometimes blessings in disguise, since they can spoil a kernel's smoothing and make problems well-conditioned.

In §19.4 – §19.7 we face up to the issues of inverse problems. Section 19.4 is an introduction to this large subject.

We should note here that wavelet transforms, already discussed in §13.10, are applicable not only to data compression and signal processing, but can also be used to transform some classes of integral equations into sparse linear problems that allow fast solution. You may wish to review §13.10 as part of reading this chapter.

Some subjects, such as *integro-differential equations*, we must simply declare to be beyond our scope. For a review of methods for integro-differential equations, see Brunner [4].

It should go without saying that this one short chapter can only barely touch on a few of the most basic methods involved in this complicated subject.

CITED REFERENCES AND FURTHER READING:

Delves, L.M., and Mohamed, J.L. 1985, *Computational Methods for Integral Equations* (Cambridge, UK: Cambridge University Press).[1]

Linz, P. 1985, *Analytical and Numerical Methods for Volterra Equations* (Philadelphia: S.I.A.M.).[2]

Atkinson, K.E. 1976, *A Survey of Numerical Methods for the Solution of Fredholm Integral Equations of the Second Kind* (Philadelphia: S.I.A.M.).[3]

Brunner, H. 1988, in *Numerical Analysis 1987*, Pitman Research Notes in Mathematics vol. 170, D.F. Griffiths and G.A. Watson, eds. (Harlow, Essex, UK: Longman Scientific and Technical), pp. 18–38.[4]

Smithies, F. 1958, *Integral Equations* (Cambridge, UK: Cambridge University Press).

Kanwal, R.P. 1971, *Linear Integral Equations* (New York: Academic Press).

Green, C.D. 1969, *Integral Equation Methods* (New York: Barnes & Noble).

19.1 Fredholm Equations of the Second Kind

We desire a numerical solution for $f(t)$ in the equation

$$f(t) = \lambda \int_a^b K(t,s)\, f(s)\, ds + g(t) \tag{19.1.1}$$

The method we describe, a very basic one, is called the *Nystrom method*. It requires the choice of some approximate *quadrature rule*:

$$\int_a^b y(s)\, ds = \sum_{j=0}^{N-1} w_j\, y(s_j) \tag{19.1.2}$$

Here the set $\{w_j\}$ are the weights of the quadrature rule, while the N points $\{s_j\}$ are the abscissas.

What quadrature rule should we use? It is certainly possible to solve integral equations with low-order quadrature rules like the extended trapezoidal or Simpson's

rules. We will see, however, that the solution method involves $O(N^3)$ operations, and so the most efficient methods tend to use high-order quadrature rules to keep N as small as possible. For smooth, nonsingular problems, nothing beats Gaussian quadrature (e.g., Gauss-Legendre quadrature, §4.6). (For nonsmooth or singular kernels, see §19.3.)

Delves and Mohamed [1] investigated methods more complicated than the Nystrom method. For straightforward Fredholm equations of the second kind, they concluded "... the clear winner of this contest has been the Nystrom routine ... with the N-point Gauss-Legendre rule. This routine is extremely simple.... Such results are enough to make a numerical analyst weep."

If we apply the quadrature rule (19.1.2) to equation (19.1.1), we get

$$f(t) = \lambda \sum_{j=0}^{N-1} w_j K(t, s_j) f(s_j) + g(t) \tag{19.1.3}$$

Evaluate equation (19.1.3) at the quadrature points:

$$f(t_i) = \lambda \sum_{j=0}^{N-1} w_j K(t_i, s_j) f(s_j) + g(t_i) \tag{19.1.4}$$

Let f_i be the vector $f(t_i)$, g_i the vector $g(t_i)$, K_{ij} the matrix $K(t_i, s_j)$, and define

$$\widetilde{K}_{ij} = K_{ij} w_j \tag{19.1.5}$$

Then, in matrix notation, equation (19.1.4) becomes

$$(\mathbf{1} - \lambda \widetilde{\mathbf{K}}) \cdot \mathbf{f} = \mathbf{g} \tag{19.1.6}$$

This is a set of N linear algebraic equations in N unknowns that can be solved by standard triangular decomposition techniques (§2.3) — that is where the $O(N^3)$ operations count comes in. The solution is usually well-conditioned, unless λ is very close to an eigenvalue.

Having obtained the solution at the quadrature points $\{t_i\}$, how do you get the solution at some other point t? You do *not* simply use polynomial interpolation. This destroys all the accuracy you have worked so hard to achieve. Nystrom's key observation was that you should use equation (19.1.3) as an interpolatory formula, maintaining the accuracy of the solution.

Our routine for solving linear Fredholm equations of the second kind is coded as the object `Fred2`. The constructor sets up equation (19.1.6) and then solves it by LU decomposition with `LUdcmp`. The Gauss-Legendre quadrature is implemented by first getting the weights and abscissas with a call to `gauleg`. The routine `Fred2` requires that you provide an external function or functor that returns $g(t)$ and another that returns λK_{ij}. It then computes the solution f at the quadrature points in the member variable `f`. It also stores the quadrature points and weights. These are used by the member function `fredin` to carry out the Nystrom interpolation of equation (19.1.3) and return the value of f at any point in the interval $[a, b]$.

To be sure the usage is clear, here is the calling sequence when you have coded the external routines as functions:

```
Doub g(const Doub t) { ... }
Doub ak(const Doub t, const Doub s) { ... }
    ...
Fred2<Doub (Doub), Doub (Doub,Doub)> fred2(a,b,n,g,ak);
Doub ans=fred2.fredin(x);
```

If the external routines are functors `Gfunc` and `Kernel`, say, then the declarations are

```
Gfunc g;                          This could have arguments if you like.
Kernel ak;
Fred2<Gfunc, Kernel> fred2(a,b,n,g,ak);
```

Here is the routine:

fred2.h

```
template <class G, class K>
struct Fred2 {
```

Solves a linear Fredholm equation of the second kind.

```
    const Doub a,b;
    const Int n;
    G &g;
    K &ak;
    VecDoub t,f,w;
    Fred2(const Doub aa, const Doub bb, const Int nn, G &gg, K &akk) :
        a(aa), b(bb), n(nn), g(gg), ak(akk), t(n), f(n), w(n)
```

Quantities `a` and `b` are input as the limits of integration. The quantity `n` is the number of points to use in the Gaussian quadrature. `g` and `ak` are user-supplied functions or functors that respectively return $g(t)$ and $\lambda K(t,s)$. This constructor computes arrays `t[0..n-1]` and `f[0..n-1]` containing the abscissas t_i of the Gaussian quadrature and the solution f at these abscissas. Also computed is the array `w[0..n-1]` of Gaussian weights for use with the Nystrom interpolation routine `fredin`.

```
    {
        MatDoub omk(n,n);
        gauleg(a,b,t,w);              Replace gauleg with another routine if not using
        for (Int i=0;i<n;i++) {           Gauss-Legendre quadrature.
            for (Int j=0;j<n;j++)     Form 1 − λK̃.
                omk[i][j]=Doub(i == j)-ak(t[i],t[j])*w[j];
            f[i]=g(t[i]);
        }
        LUdcmp alu(omk);              Solve linear equations.
        alu.solve(f,f);
    }

    Doub fredin(const Doub x)
```

Given arrays `t[0..n-1]` and `w[0..n-1]` containing the abscissas and weights of the Gaussian quadrature, and given the solution array `f[0..n-1]`, this function returns the value of f at `x` using the Nystrom interpolation formula.

```
    {
        Doub sum=0.0;
        for (Int i=0;i<n;i++) sum += ak(x,t[i])*w[i]*f[i];
        return g(x)+sum;
    }
};
```

One disadvantage of a method based on Gaussian quadrature is that there is no simple way to obtain an estimate of the error in the result. The best practical method is to increase N by 50%, say, and treat the difference between the two estimates as a conservative estimate of the error in the result obtained with the larger value of N.

Turn now to solutions of the homogeneous equation. If we set $\lambda = 1/\sigma$ and $\mathbf{g} = 0$, then equation (19.1.6) becomes a standard eigenvalue equation,

$$\tilde{\mathbf{K}} \cdot \mathbf{f} = \sigma \mathbf{f} \tag{19.1.7}$$

which we can solve with any convenient matrix eigenvalue routine (see Chapter 11). Note that if our original problem had a symmetric kernel, then the matrix $\mathbf{K}$ would be symmetric. However, since the weights w_j are not equal for most quadrature rules, the matrix $\tilde{\mathbf{K}}$ (equation 19.1.5) is not symmetric. The matrix eigenvalue problem is much easier for symmetric matrices, and so we should restore the symmetry if possible. Provided the weights are positive (which they are for Gaussian quadrature), we can define the diagonal matrix $\mathbf{D} = \text{diag}(w_j)$ and its square root, $\mathbf{D}^{1/2} = \text{diag}(\sqrt{w_j})$. Then equation (19.1.7) becomes

$$\mathbf{K} \cdot \mathbf{D} \cdot \mathbf{f} = \sigma \mathbf{f}$$

Multiplying by $\mathbf{D}^{1/2}$, we get

$$\left(\mathbf{D}^{1/2} \cdot \mathbf{K} \cdot \mathbf{D}^{1/2}\right) \cdot \mathbf{h} = \sigma \mathbf{h} \tag{19.1.8}$$

where $\mathbf{h} = \mathbf{D}^{1/2} \cdot \mathbf{f}$. Equation (19.1.8) is now in the form of a symmetric eigenvalue problem.

Solution of equations (19.1.7) or (19.1.8) will in general give N eigenvalues, where N is the number of quadrature points used. For square-integrable kernels, these will provide good approximations to the lowest N eigenvalues of the integral equation. Kernels of *finite rank* (also called *degenerate* or *separable* kernels) have only a finite number of nonzero eigenvalues (possibly none). You can diagnose this situation by a cluster of eigenvalues σ that are zero to machine precision. The number of nonzero eigenvalues will stay constant as you increase N to improve their accuracy. Some care is required here: A nondegenerate kernel can have an infinite number of eigenvalues that have an accumulation point at $\sigma = 0$. You distinguish the two cases by the behavior of the solution as you increase N. If you suspect a degenerate kernel, you will usually be able to solve the problem by analytic techniques described in all the textbooks.

CITED REFERENCES AND FURTHER READING:

Delves, L.M., and Mohamed, J.L. 1985, *Computational Methods for Integral Equations* (Cambridge, UK: Cambridge University Press).[1]

Atkinson, K.E. 1976, *A Survey of Numerical Methods for the Solution of Fredholm Integral Equations of the Second Kind* (Philadelphia: S.I.A.M.).

19.2 Volterra Equations

Let us now turn to Volterra equations, of which our prototype is the Volterra equation of the second kind,

$$f(t) = \int_a^t K(t,s) f(s)\, ds + g(t) \tag{19.2.1}$$

Most algorithms for Volterra equations march out from $t = a$, building up the solution as they go. In this sense they resemble not only forward substitution (as discussed in §19.0), but also initial value problems for ordinary differential equations. In fact, many algorithms for ODEs have counterparts for Volterra equations.

The simplest way to proceed is to solve the equation on a mesh with uniform spacing:

$$t_i = a + ih, \quad i = 0, 1, \ldots, N, \qquad h \equiv \frac{b-a}{N} \tag{19.2.2}$$

To do so, we must choose a quadrature rule. For a uniform mesh, the simplest scheme is the trapezoidal rule, equation (4.1.11):

$$\int_a^{t_i} K(t_i, s) f(s)\, ds = h\left(\tfrac{1}{2}K_{i0} f_0 + \sum_{j=1}^{i-1} K_{ij} f_j + \tfrac{1}{2}K_{ii} f_i\right) \tag{19.2.3}$$

Thus the trapezoidal method for equation (19.2.1) is

$$\begin{aligned} f_0 &= g_0 \\ (1 - \tfrac{1}{2}hK_{ii})\, f_i &= h\left(\tfrac{1}{2}K_{i0} f_0 + \sum_{j=1}^{i-1} K_{ij} f_j\right) + g_i, \qquad i = 1, \ldots, N \end{aligned} \tag{19.2.4}$$

(For a Volterra equation of the first kind, the leading 1 on the left would be absent, and g would have opposite sign, with corresponding straightforward changes in the rest of the discussion.)

Equation (19.2.4) is an explicit prescription that gives the solution in $O(N^2)$ operations. Unlike Fredholm equations, it is not necessary to solve a system of linear equations. Volterra equations thus usually involve less work than the corresponding Fredholm equations, which, as we have seen, do involve the inversion of, sometimes large, linear systems.

The efficiency of solving Volterra equations is somewhat counterbalanced by the fact that *systems* of these equations occur more frequently in practice. If we interpret equation (19.2.1) as a *vector* equation for the vector of m functions $f(t)$, then the kernel $K(t, s)$ is an $m \times m$ matrix. Equation (19.2.4) must now also be understood as a vector equation. For each i, we have to solve the $m \times m$ set of linear algebraic equations by Gaussian elimination.

The routine `voltra` below implements this algorithm. You must supply an external function or functor that returns the kth function of the vector $g(t)$ at the point t and another that returns the (k, l) element of the matrix $K(t, s)$ at (t, s). The routine `voltra` then returns the vector $f(t)$ at the regularly spaced points t_i.

voltra.h

```
template <class G, class K>
void voltra(const Doub t0, const Doub h, G &g, K &ak, VecDoub_O &t, MatDoub_O &f)
```

Solves a set of `m` linear Volterra equations of the second kind using the extended trapezoidal rule. On input, `t0` is the starting point of the integration and `h` is the stepsize. `g(k,t)` is a user-supplied function or functor that returns $g_k(t)$, while `ak(k,l,t,s)` is another user-supplied function or functor that returns the (k, l) element of the matrix $K(t, s)$. The solution is returned in `f[0..m-1][0..n-1]`, with the corresponding abscissas in `t[0..n-1]`, where `n-1` is the number of steps to be taken. The value of `m` is determined from the row-dimension of the solution matrix `f`.

```
{
    Int m=f.nrows();
```

```
    Int n=f.ncols();
    VecDoub b(m);
    MatDoub a(m,m);
    t[0]=t0;
    for (Int k=0;k<m;k++) f[k][0]=g(k,t[0]);          Initialize.
    for (Int i=1;i<n;i++) {                           Take a step h.
        t[i]=t[i-1]+h;
        for (Int k=0;k<m;k++) {
            Doub sum=g(k,t[i]);                       Accumulate right-hand side of linear
            for (Int l=0;l<m;l++) {                       equations in sum.
                sum += 0.5*h*ak(k,l,t[i],t[0])*f[l][0];
                for (Int j=1;j<i;j++)
                    sum += h*ak(k,l,t[i],t[j])*f[l][j];
                if (k == l)                           Left-hand side goes in matrix a.
                    a[k][l]=1.0-0.5*h*ak(k,l,t[i],t[i]);
                else
                    a[k][l] = -0.5*h*ak(k,l,t[i],t[i]);
            }
            b[k]=sum;
        }
        LUdcmp alu(a);                                Solve linear equations.
        alu.solve(b,b);
        for (Int k=0;k<m;k++) f[k][i]=b[k];
    }
}
```

For nonlinear Volterra equations, equation (19.2.4) holds with the product $K_{ii}\,f_i$ replaced by $K_{ii}(f_i)$, and similarly for the other two products of K's and f's. Thus, for each i we solve a nonlinear equation for f_i with a known right-hand side. Newton's method (§9.4 or §9.6) with an initial guess of f_{i-1} usually works very well provided the stepsize is not too big.

Higher-order methods for solving Volterra equations are, in our opinion, not as important as for Fredholm equations, since Volterra equations are relatively easy to solve. However, there is an extensive literature on the subject. Several difficulties arise. First, any method that achieves higher order by operating on several quadrature points simultaneously will need a special method to get started, when values at the first few points are not yet known.

Second, stable quadrature rules can give rise to unexpected instabilities in integral equations. For example, suppose we try to replace the trapezoidal rule in the algorithm above with Simpson's rule. Simpson's rule naturally integrates over an interval $2h$, so we easily get the function values at the even mesh points. For the odd mesh points, we could try appending one panel of the trapezoidal rule. But to which end of the integration should we append it? We could do one step of the trapezoidal rule followed by all Simpson's rule, or Simpson's rule with one step of the trapezoidal rule at the end. Surprisingly, the former scheme is unstable, while the latter is fine!

A simple approach that can be used with the trapezoidal method given above is Richardson extrapolation: Compute the solution with stepsizes h and $h/2$. Then, assuming the error scales with h^2, compute

$$f_{\rm E} = \frac{4f(h/2) - f(h)}{3} \tag{19.2.5}$$

This procedure can be repeated as with Romberg integration.

The general consensus is that the best of the higher-order methods is the *block-by-block method* (see [1]). Another important topic is the use of variable stepsize

methods, which are much more efficient if there are sharp features in K or f. Variable stepsize methods are quite a bit more complicated than their counterparts for differential equations; we refer you to the literature [1,2] for a discussion.

You should also be on the lookout for singularities in the integrand. If you find them, look to §19.3 for additional ideas.

CITED REFERENCES AND FURTHER READING:

Linz, P. 1985, *Analytical and Numerical Methods for Volterra Equations* (Philadelphia: S.I.A.M.).[1]

Delves, L.M., and Mohamed, J.L. 1985, *Computational Methods for Integral Equations* (Cambridge, UK: Cambridge University Press).[2]

19.3 Integral Equations with Singular Kernels

Many integral equations have singularities in either the kernel or the solution or both. A simple quadrature method will show poor convergence with N if such singularities are ignored. There is sometimes art in how singularities are best handled.

We start with a few straightforward suggestions:

1. Integrable singularities can often be removed by a change of variable. For example, the singular behavior $K(t,s) \sim s^{1/2}$ or $s^{-1/2}$ near $s = 0$ can be removed by the transformation $z = s^{1/2}$. Note that we are assuming that the singular behavior is confined to K, whereas the quadrature actually involves the product $K(t,s)f(s)$, and it is this product that must be "fixed." Ideally, you must deduce the singular nature of the product before you try a numerical solution, and take the appropriate action. Commonly, however, a singular kernel does *not* produce a singular solution $f(t)$. (The highly singular kernel $K(t,s) = \delta(t-s)$ is simply the identity operator, for example.)

2. If $K(t,s)$ can be factored as $w(s)\bar{K}(t,s)$, where $w(s)$ is singular and $\bar{K}(t,s)$ is smooth, then a Gaussian quadrature based on $w(s)$ as a weight function will work well. Even if the factorization is only approximate, the convergence is often improved dramatically. All you have to do is replace `gauleg` in the routine `fred2` by another quadrature routine. Section 4.6 explained how to construct such quadratures; or you can find tabulated abscissas and weights in the standard references [1,2]. You must of course supply $\bar{K}$ instead of K.

This method is a special case of the *product Nystrom method* [3,4], where one factors out a singular term $p(t,s)$ depending on both t and s from K and constructs suitable weights for its Gaussian quadrature. The calculations in the general case are quite cumbersome, because the weights depend on the chosen $\{t_i\}$ as well as the form of $p(t,s)$.

We prefer to implement the product Nystrom method on a uniform grid, with a quadrature scheme that generalizes the extended Simpson's 3/8 rule (equation 4.1.5) to arbitrary weight functions. We discuss this in the subsections below.

3. Special quadrature formulas are also useful when the kernel is not strictly singular, but is "almost" so. One example is when the kernel is concentrated near $t = s$ on a scale much smaller than the scale on which the solution $f(t)$ varies. In that case, a quadrature formula can be based on locally approximating $f(s)$ by a polynomial or spline, while calculating the first few *moments* of the kernel $K(t,s)$ at the tabulation points t_i. In such a scheme the narrow width of the kernel becomes an asset, rather than a liability: The quadrature becomes exact as the width of the kernel goes to zero.

4. An infinite range of integration is also a form of singularity. Truncating the range at a large finite value should be used only as a last resort. If the kernel goes rapidly to zero, then a Gauss-Laguerre [$w \sim \exp(-\alpha s)$] or Gauss-Hermite [$w \sim \exp(-s^2)$] quadrature should work well. Long-tailed functions often succumb to the transformation

$$s = \frac{2\alpha}{z+1} - \alpha \tag{19.3.1}$$

which maps $0 < s < \infty$ to $1 > z > -1$ so that Gauss-Legendre integration can be used. Here $\alpha > 0$ is a constant that you adjust to improve the convergence.

5. A common situation in practice is that $K(t,s)$ is singular along the diagonal line $t = s$. Here the Nystrom method fails completely because the kernel gets evaluated at (t_i, s_i). *Subtraction of the singularity* is one possible cure:

$$\begin{aligned}\int_a^b K(t,s)\,f(s)\,ds &= \int_a^b K(t,s)[f(s)-f(t)]\,ds + \int_a^b K(t,s)\,f(t)\,ds \\ &= \int_a^b K(t,s)[f(s)-f(t)]\,ds + r(t)\,f(t)\end{aligned} \tag{19.3.2}$$

where $r(t) = \int_a^b K(t,s)\,ds$ is computed analytically or numerically. If the first term on the right-hand side is now regular, we can use the Nystrom method. Instead of equation (19.1.4), we get

$$f_i = \lambda \sum_{\substack{j=0 \\ j\neq i}}^{N-1} w_j K_{ij}[f_j - f_i] + \lambda r_i f_i + g_i \tag{19.3.3}$$

Sometimes the subtraction process must be repeated before the kernel is completely regularized. See [3] for details. (And read on for a different, we think better, way to handle diagonal singularities.)

19.3.1 Quadrature on a Uniform Mesh with Arbitrary Weight

It is possible in general to find n-point linear quadrature rules that approximate the integral of a function $f(x)$, times an arbitrary weight function $w(x)$, over an arbitrary range of integration (a,b), as the sum of weights times n evenly spaced values of the function $f(x)$, say at $x = kh, (k+1)h, \ldots, (k+n-1)h$. The general scheme for deriving such quadrature rules is to write down the n linear equations that must be satisfied if the quadrature rule is to be exact for the n functions $f(x) = \text{const}, x, x^2, \ldots, x^{n-1}$, and then solve these for the coefficients. This can be done analytically, once and for all, if the moments of the weight function over the same range of integration,

$$W_n \equiv \frac{1}{h^n}\int_a^b x^n w(x)dx \tag{19.3.4}$$

are assumed to be known. Here the prefactor h^{-n} is chosen to make W_n scale as h if (as in the usual case) $b-a$ is proportional to h.

Carrying out this prescription for the four-point case gives the result

$$\begin{aligned}&\int_a^b w(x)\,f(x)dx = \\ &\quad \tfrac{1}{6}f(kh)\big[(k+1)(k+2)(k+3)W_0 - (3k^2+12k+11)W_1 + 3(k+2)W_2 - W_3\big] \\ &\quad + \tfrac{1}{2}f([k+1]h)\big[-k(k+2)(k+3)W_0 + (3k^2+10k+6)W_1 - (3k+5)W_2 + W_3\big] \\ &\quad + \tfrac{1}{2}f([k+2]h)\big[k(k+1)(k+3)W_0 - (3k^2+8k+3)W_1 + (3k+4)W_2 - W_3\big] \\ &\quad + \tfrac{1}{6}f([k+3]h)\big[-k(k+1)(k+2)W_0 + (3k^2+6k+2)W_1 - 3(k+1)W_2 + W_3\big]\end{aligned} \tag{19.3.5}$$

While the terms in brackets superficially appear to scale as k^2, there is typically cancellation at both $O(k^2)$ and $O(k)$.

Equation (19.3.5) can be specialized to various choices of (a,b). The obvious choice is $a = kh$, $b = (k+3)h$, in which case we get a four-point quadrature rule that generalizes

Simpson's 3/8 rule (equation 4.1.5). In fact, we can recover this special case by setting $w(x) = 1$, in which case (19.3.4) becomes

$$W_n = \frac{h}{n+1}[(k+3)^{n+1} - k^{n+1}] \tag{19.3.6}$$

The four terms in square brackets in equation (19.3.5) each become independent of k, and (19.3.5) in fact reduces to

$$\int_{kh}^{(k+3)h} f(x)dx = \frac{3h}{8}f(kh) + \frac{9h}{8}f([k+1]h) + \frac{9h}{8}f([k+2]h) + \frac{3h}{8}f([k+3]h) \tag{19.3.7}$$

Back to the case of general $w(x)$, some other choices for a and b are also useful. For example, we may want to choose (a, b) to be $([k+1]h, [k+3]h)$ or $([k+2]h, [k+3]h)$, allowing us to finish off an extended rule whose number of intervals is not a multiple of three, without loss of accuracy: The integral will be estimated using the four values $f(kh), \ldots, f([k+3]h)$. Even more useful is to choose (a, b) to be $([k+1]h, [k+2]h)$, thus using four points to integrate a centered single interval. These weights, when sewed together into an extended formula, give quadrature schemes that have smooth coefficients, i.e., without the Simpson-like $2, 4, 2, 4, 2$ alternation. (In fact, this was the technique that we used to derive equation 4.1.14, which you may now wish to re-examine.)

All these rules are of the same order as the extended Simpson's rule, that is, exact for $f(x)$ a cubic polynomial. Rules of lower order, if desired, are similarly obtained. The three-point formula is

$$\begin{aligned}\int_a^b w(x)f(x)dx = {} & \tfrac{1}{2}f(kh)\big[(k+1)(k+2)W_0 - (2k+3)W_1 + W_2\big] \\ & + f([k+1]h)\big[-k(k+2)W_0 + 2(k+1)W_1 - W_2\big] \\ & + \tfrac{1}{2}f([k+2]h)\big[k(k+1)W_0 - (2k+1)W_1 + W_2\big]\end{aligned} \tag{19.3.8}$$

Here the simple special case is to take $w(x) = 1$, so that

$$W_n = \frac{h}{n+1}[(k+2)^{n+1} - k^{n+1}] \tag{19.3.9}$$

Then equation (19.3.8) becomes Simpson's rule,

$$\int_{kh}^{(k+2)h} f(x)dx = \frac{h}{3}f(kh) + \frac{4h}{3}f([k+1]h) + \frac{h}{3}f([k+2]h) \tag{19.3.10}$$

For nonconstant weight functions $w(x)$, however, equation (19.3.8) gives rules of one order less than Simpson, since they do not benefit from the extra symmetry of the constant case.

The two-point formula is simply

$$\int_{kh}^{(k+1)h} w(x)f(x)dx = f(kh)[(k+1)W_0 - W_1] + f([k+1]h)[-kW_0 + W_1] \tag{19.3.11}$$

Here is a routine `Wwghts` that uses the above formulas to compute an extended N-point quadrature rule for the interval $(a, b) = (0, [N-1]h)$. Input to `Wwghts` is a user-supplied object called `quad` in the routine. This object must contain a function `kermom`, which is called to get the first four *indefinite-integral* moments of $w(x)$, namely

$$F_m(y) \equiv \int^y s^m w(s)ds \qquad m = 0, 1, 2, 3 \tag{19.3.12}$$

(The lower limit is arbitrary and can be chosen for convenience.) Cautionary note: When called with $N < 4$, `Wwghts` returns a rule of lower order than Simpson; you should structure your problem to avoid this.

fred_singular.h

```
template <class Q>
struct Wwghts {
```

Constructs weights for the n-point equal-interval quadrature from 0 to (n−1)h of a function $f(x)$ times an arbitrary (possibly singular) weight function $w(x)$. The indefinite-integral moments $F_n(y)$ of $w(x)$ are provided by the user-supplied function kermom in the quad object.

```
    Doub h;
    Int n;
    Q &quad;
    VecDoub wghts;
    Wwghts(Doub hh, Int nn, Q &q) : h(hh), n(nn), quad(q), wghts(n) {}
```

Constructor arguments are h, n, and the user-supplied quad object.

```
    VecDoub weights()
```

This function returns the weights in wghts[0..n-1].

```
    {
        Int k;
        Doub fac;
        Doub hi=1.0/h;
        for (Int j=0;j<n;j++)                    Zero all the weights so we can sum into
            wghts[j]=0.0;                            them.
        if (n >= 4) {                            Use highest available order.
            VecDoub wold(4),wnew(4),w(4);
            wold=quad.kermom(0.0);               Evaluate indefinite integrals at lower end.
            Doub b=0.0;                          For another problem, you might change
            for (Int j=0;j<n-3;j++) {                this lower limit.
                Doub c=j;                        This is called k in equation (19.3.5).
                Doub a=b;                        Set upper and lower limits for this step.
                b=a+h;
                if (j == n-4) b=(n-1)*h;         Last interval: Go all the way to end.
                wnew=quad.kermom(b);
                for (fac=1.0,k=0;k<4;k++,fac*=hi)            Equation (19.3.4).
                    w[k]=(wnew[k]-wold[k])*fac;
                wghts[j] += (((c+1.0)*(c+2.0)*(c+3.0)*w[0]   Equation (19.3.5).
                    -(11.0+c*(12.0+c*3.0))*w[1]+3.0*(c+2.0)*w[2]-w[3])/6.0);
                wghts[j+1] += ((-c*(c+2.0)*(c+3.0)*w[0]
                    +(6.0+c*(10.0+c*3.0))*w[1]-(3.0*c+5.0)*w[2]+w[3])*0.5);
                wghts[j+2] += ((c*(c+1.0)*(c+3.0)*w[0]
                    -(3.0+c*(8.0+c*3.0))*w[1]+(3.0*c+4.0)*w[2]-w[3])*0.5);
                wghts[j+3] += ((-c*(c+1.0)*(c+2.0)*w[0]
                    +(2.0+c*(6.0+c*3.0))*w[1]-3.0*(c+1.0)*w[2]+w[3])/6.0);
                for (k=0;k<4;k++) wold[k]=wnew[k];    Reset lower limits for moments.
            }
        } else if (n == 3) {                     Lower-order cases; not recommended.
            VecDoub wold(3),wnew(3),w(3);
            wold=quad.kermom(0.0);
            wnew=quad.kermom(h+h);
            w[0]=wnew[0]-wold[0];
            w[1]=hi*(wnew[1]-wold[1]);
            w[2]=hi*hi*(wnew[2]-wold[2]);
            wghts[0]=w[0]-1.5*w[1]+0.5*w[2];
            wghts[1]=2.0*w[1]-w[2];
            wghts[2]=0.5*(w[2]-w[1]);
        } else if (n == 2) {
            VecDoub wold(2),wnew(2),w(2);
            wold=quad.kermom(0.0);
            wnew=quad.kermom(h);
            wghts[0]=wnew[0]-wold[0]-(wghts[1]=hi*(wnew[1]-wold[1]));
        }
        return wghts;
    }
};
```

We will now give an example of how to apply Wwghts to a singular integral equation.

19.3.2 Worked Example: A Diagonally Singular Kernel

As a particular example, consider the integral equation

$$f(x) + \int_0^\pi K(x,y) f(y) dy = \sin x \tag{19.3.13}$$

with the (arbitrarily chosen) nasty kernel

$$K(x,y) = \cos x \cos y \times \begin{cases} -\ln(x-y) & y < x \\ \sqrt{y-x} & y \geq x \end{cases} \tag{19.3.14}$$

which has a logarithmic singularity on the left of the diagonal, combined with a square-root discontinuity on the right.

The first step is to do (analytically, in this case) the required moment integrals over the singular part of the kernel, equation (19.3.12). Since these integrals are done at a fixed value of x, we can use x as the lower limit. For any specified value of y, the required indefinite integral is then either

$$F_m(y;x) = \int_x^y s^m (s-x)^{1/2} ds = \int_0^{y-x} (x+t)^m t^{1/2} dt \qquad \text{if } y > x \tag{19.3.15}$$

or

$$F_m(y;x) = -\int_x^y s^m \ln(x-s) ds = \int_0^{x-y} (x-t)^m \ln t \, dt \qquad \text{if } y < x \tag{19.3.16}$$

(where a change of variable has been made in the second equality in each case). Doing these integrals analytically (e.g., using a symbolic integration package), we package the resulting formulas in the function `kermom` in the following routine, `Quad_matrix`. Note that `w`$(j+1)$ returns $F_j(y;x)$. The constructor of `Quad_matrix` calls `Wwghts` to get the quadrature weights and then constructs the quadrature matrix.

fred_singular.h

```
struct Quad_matrix {
```
Constructs in `a[0..n-1][0..n-1]` the quadrature matrix for an example Fredholm equation of the second kind.
```
    Int n;
    Doub x;                          Communicates with kermom.
    Quad_matrix(MatDoub_O &a) : n(a.nrows())
```
The constructor obtains the quadrature weights that integrate the singular part of the kernel via calls to `Wwghts`. It then sums the weights with the nonsingular part of the kernel to obtain the quadrature matrix.
```
    {
        const Doub PI=3.14159263589793238;
        VecDoub wt(n);
        Doub h=PI/(n-1);
        Wwghts<Quad_matrix> w(h,n,*this);
        for (Int j=0;j<n;j++) {
            x=j*h;                   Set x for kermom.
            wt=w.weights();
            Doub cx=cos(x);          Part of nonsingular kernel.
            for (Int k=0;k<n;k++)    Put together all the pieces of the kernel.
                a[j][k]=wt[k]*cx*cos(k*h);
            ++a[j][j];               For equations of the second kind, there is a diagonal
        }                                piece independent of h.
    }
    VecDoub kermom(const Doub y)
```
Returns `w[0..m-1]`, the first `m` indefinite-integral moments of one row of the singular part of the kernel. (For this example, `m` is hard-wired to be 4.) The input variable `y` labels the column, while the member variable `x` is the row. We can take `x` as the lower limit of integration. Thus, we return the moment integrals either purely to the left or purely to the right of the diagonal.
```
    {
```

```
        Doub d,df,clog,x2,x3,x4,y2;
        VecDoub w(4);
        if (y >= x) {
            d=y-x;
            df=2.0*sqrt(d)*d;
            w[0]=df/3.0;
            w[1]=df*(x/3.0+d/5.0);
            w[2]=df*((x/3.0 + 0.4*d)*x + d*d/7.0);
            w[3]=df*(((x/3.0 + 0.6*d)*x + 3.0*d*d/7.0)*x+d*d*d/9.0);
        } else {
            x3=(x2=x*x)*x;
            x4=x2*x2;
            y2=y*y;
            d=x-y;
            w[0]=d*((clog=log(d))-1.0);
            w[1] = -0.25*(3.0*x+y-2.0*clog*(x+y))*d;
            w[2]=(-11.0*x3+y*(6.0*x2+y*(3.0*x+2.0*y))
                +6.0*clog*(x3-y*y2))/18.0;
            w[3]=(-25.0*x4+y*(12.0*x3+y*(6.0*x2+y*
                (4.0*x+3.0*y)))+12.0*clog*(x4-(y2*y2)))/48.0;
        }
        return w;
    }
};
```

Finally, we solve the linear system for any particular right-hand side, here $\sin x$.

fred_singular.h

```
Int main_fredex(void)
```

This sample program shows how to solve a Fredholm equation of the second kind using the product Nystrom method and a quadrature rule especially constructed for a particular, singular, kernel.

```
{
    const Int N=40;                 Here the size of the grid is specified.
    const Doub PI=3.141592653589793238;
    VecDoub g(N);
    MatDoub a(N,N);
    Quad_matrix qmx(a);             Make the quadrature matrix; all the action is here.
    LUdcmp alu(a);                  Decompose the matrix.
    for (Int j=0;j<N;j++)           Construct the right-hand side, here sin x.
        g[j]=sin(j*PI/(N-1));
    alu.solve(g,g);                 Backsubstitute.
    for (Int j=0;j<N;j++) {         Write out the solution.
        Doub x=j*PI/(N-1);
        cout << fixed << setprecision(2) << setw(6) << (j+1);
        cout << setprecision(6) << setw(13) << x << setw(13) << g[j] << endl;
    }
    return 0;
}
```

With $N = 40$, this program gives accuracy at about the 10^{-5} level. The accuracy increases as N^4 (as it should for our Simpson-order quadrature scheme) *despite* the highly singular kernel. Figure 19.3.1 shows the solution obtained, also plotting the solution for smaller values of N, which are themselves seen to be remarkably faithful. Notice that the solution is smooth, even though the kernel is singular, a common occurrence.

CITED REFERENCES AND FURTHER READING:

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at `http://www.nr.com/aands`.[1]

Stroud, A.H., and Secrest, D. 1966, *Gaussian Quadrature Formulas* (Englewood Cliffs, NJ: Prentice-Hall).[2]

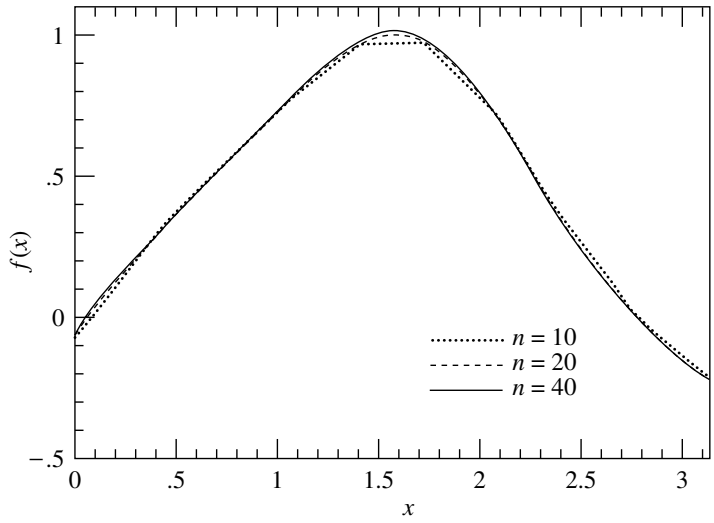


Figure 19.3.1. Solution of the example integral equation (19.3.14) with grid sizes $N = 10$, 20, and 40. The tabulated solution values have been connected by straight lines; in practice, one would interpolate a small N solution more smoothly.

Delves, L.M., and Mohamed, J.L. 1985, *Computational Methods for Integral Equations* (Cambridge, UK: Cambridge University Press).[3]

Atkinson, K.E. 1976, *A Survey of Numerical Methods for the Solution of Fredholm Integral Equations of the Second Kind* (Philadelphia: S.I.A.M.).[4]

19.4 Inverse Problems and the Use of A Priori Information

Later discussion will be facilitated by some preliminary mention of a couple of mathematical points. Suppose that $\mathbf{u}$ is an "unknown" vector that we plan to determine by some minimization principle. Let $\mathcal{A}[\mathbf{u}] > 0$ and $\mathcal{B}[\mathbf{u}] > 0$ be two positive functionals of $\mathbf{u}$, so that we can try to determine $\mathbf{u}$ by either

$$\text{minimize:} \quad \mathcal{A}[\mathbf{u}] \qquad \text{or} \qquad \text{minimize:} \quad \mathcal{B}[\mathbf{u}] \tag{19.4.1}$$

(Of course these will generally give different answers for $\mathbf{u}$.) As another possibility, now suppose that we want to minimize $\mathcal{A}[\mathbf{u}]$ subject to the *constraint* that $\mathcal{B}[\mathbf{u}]$ have some particular value, say b. The method of Lagrange multipliers gives the variation

$$\frac{\delta}{\delta \mathbf{u}} \{\mathcal{A}[\mathbf{u}] + \lambda_1(\mathcal{B}[\mathbf{u}] - b)\} = \frac{\delta}{\delta \mathbf{u}} (\mathcal{A}[\mathbf{u}] + \lambda_1 \mathcal{B}[\mathbf{u}]) = 0 \tag{19.4.2}$$

where λ_1 is a Lagrange multiplier. Notice that b is absent in the second equality, since it doesn't depend on $\mathbf{u}$.

Next, suppose that we change our minds and decide to minimize $\mathcal{B}[\mathbf{u}]$ subject to the constraint that $\mathcal{A}[\mathbf{u}]$ have a particular value, a. Instead of equation (19.4.2) we

have

$$\frac{\delta}{\delta \mathbf{u}}\{\mathcal{B}[\mathbf{u}] + \lambda_2(\mathcal{A}[\mathbf{u}] - a)\} = \frac{\delta}{\delta \mathbf{u}}(\mathcal{B}[\mathbf{u}] + \lambda_2 \mathcal{A}[\mathbf{u}]) = 0 \tag{19.4.3}$$

with, this time, λ_2 the Lagrange multiplier. Multiplying equation (19.4.3) by the constant $1/\lambda_2$, and identifying $1/\lambda_2$ with λ_1, we see that the actual variations are exactly the same in the two cases. Both cases will yield the same one-parameter family of solutions, say $\mathbf{u}(\lambda_1)$. As λ_1 varies from 0 to ∞, the solution $\mathbf{u}(\lambda_1)$ varies along a so-called *trade-off curve* between the problem of minimizing $\mathcal{A}$ and the problem of minimizing $\mathcal{B}$. Any solution along this curve can equally well be thought of as either (i) a minimization of $\mathcal{A}$ for some constrained value of $\mathcal{B}$, or (ii) a minimization of $\mathcal{B}$ for some constrained value of $\mathcal{A}$, or (iii) a weighted minimization of the sum $\mathcal{A} + \lambda_1 \mathcal{B}$.

The second preliminary point has to do with *degenerate* minimization principles. In the example above, now suppose that $\mathcal{A}[\mathbf{u}]$ has the particular form

$$\mathcal{A}[\mathbf{u}] = |\mathbf{A} \cdot \mathbf{u} - \mathbf{c}|^2 \tag{19.4.4}$$

for some matrix $\mathbf{A}$ and vector $\mathbf{c}$. If $\mathbf{A}$ has fewer rows than columns, or if $\mathbf{A}$ is square but degenerate (has a nontrivial nullspace; see §2.6, especially Figure 2.6.1), then minimizing $\mathcal{A}[\mathbf{u}]$ will *not* give a unique solution for $\mathbf{u}$. (To see why, review §15.4, and note that for a "design matrix" $\mathbf{A}$ with fewer rows than columns, the matrix $\mathbf{A}^T \cdot \mathbf{A}$ in the normal equations 15.4.10 is degenerate.) *However*, if we add any multiple λ times a nondegenerate quadratic form $\mathcal{B}[\mathbf{u}]$, for example $\mathbf{u} \cdot \mathbf{H} \cdot \mathbf{u}$ with $\mathbf{H}$ a positive-definite matrix, then minimization of $\mathcal{A}[\mathbf{u}] + \lambda \mathcal{B}[\mathbf{u}]$ *will* lead to a unique solution for $\mathbf{u}$. (The sum of two quadratic forms is itself a quadratic form, with the second piece guaranteeing nondegeneracy.)

We can combine these two points, for this conclusion: When a quadratic minimization principle is combined with a quadratic constraint, and both are positive, only *one* of the two need be nondegenerate for the overall problem to be well-posed. We are now equipped to face the subject of inverse problems.

19.4.1 The Inverse Problem with Zeroth-Order Regularization

Suppose that $u(x)$ is some unknown or underlying (u stands for both unknown and underlying!) physical process, which we hope to determine by a set of N measurements c_i, $i = 0, 1, \ldots, N-1$. The relation between $u(x)$ and the c_i's is that each c_i measures a (hopefully distinct) aspect of $u(x)$ through its own linear response kernel r_i, and with its own measurement error n_i. In other words,

$$c_i \equiv s_i + n_i = \int r_i(x) u(x) dx + n_i \tag{19.4.5}$$

(compare this to equations 13.3.1 and 13.3.2). Within the assumption of linearity, this is quite a general formulation. The c_i's might approximate values of $u(x)$ at certain locations x_i, in which case $r_i(x)$ would have the form of a more or less narrow instrumental response centered around $x = x_i$. Or, the c_i's might "live" in an entirely different function space from $u(x)$, measuring different Fourier components of $u(x)$, for example.

The *inverse problem* is, given the c_i's, the $r_i(x)$'s, and perhaps some information about the errors n_i such as their covariance matrix,

$$S_{ij} \equiv \text{Covar}[n_i, n_j] \tag{19.4.6}$$

how do we find a good statistical estimator of $u(x)$, call it $\hat{u}(x)$?

It should be obvious that this is an ill-posed problem. After all, how can we reconstruct a whole function $\hat{u}(x)$ from only a finite number of discrete values c_i? Yet, whether formally or informally, we do this all the time in science. We routinely measure "enough points" and then "draw a curve through them." In doing so, we are making some assumptions, either about the underlying function $u(x)$, or about the nature of the response functions $r_i(x)$, or both. Our purpose now is to formalize these assumptions, and to extend our abilities to cases where the measurements and underlying function live in quite different function spaces. (How do you "draw a curve" through a scattering of Fourier coefficients?)

We can't really want every point x of the function $\hat{u}(x)$. We do want some large number M of discrete points x_μ, $\mu = 0, 1, \ldots, M-1$, where M is sufficiently large, and the x_μ's are sufficiently evenly spaced, that neither $u(x)$ nor $r_i(x)$ varies much between any x_μ and $x_{\mu+1}$. (Here and following we will use Greek letters like μ to denote values in the space of the underlying process, and Roman letters like i to denote values of immediate observables.) For such a dense set of x_μ's, we can replace equation (19.4.5) by a quadrature like

$$c_i = \sum_\mu R_{i\mu} u(x_\mu) + n_i \tag{19.4.7}$$

where the $N \times M$ matrix $\mathbf{R}$ has components

$$R_{i\mu} \equiv r_i(x_\mu)(x_{\mu+1} - x_{\mu-1})/2 \tag{19.4.8}$$

(or any other simple quadrature — it rarely matters which). We will view equations (19.4.5) and (19.4.7) as being equivalent for practical purposes.

How do you solve a set of equations like equation (19.4.7) for the unknown $u(x_\mu)$'s? Here is a bad way, but one that contains the germ of some correct ideas: Form a χ^2 measure of how well a model $u(x)$ agrees with the measured data,

$$\begin{aligned} \chi^2 &= \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} \left[c_i - \sum_{\mu=0}^{M-1} R_{i\mu} u(x_\mu) \right] S_{ij}^{-1} \left[c_j - \sum_{\mu=0}^{M-1} R_{j\mu} u(x_\mu) \right] \\ &\approx \sum_{i=0}^{N-1} \left[\frac{c_i - \sum_{\mu=0}^{M-1} R_{i\mu} u(x_\mu)}{\sigma_i} \right]^2 \end{aligned} \tag{19.4.9}$$

(compare with equation 15.1.6). Here $\mathbf{S}^{-1}$ is the inverse of the covariance matrix, and the approximate equality holds if you can neglect the off-diagonal covariances, with $\sigma_i \equiv (\text{Covar}[i, i])^{1/2}$.

Now you can use the method of singular value decomposition (SVD) in §15.4 to find the vector $\mathbf{u}$ that minimizes equation (19.4.9). Don't try to use the method of normal equations; since M is greater than N, they will be singular, as we already discussed. The SVD process will thus surely find a large number of zero singular

values, indicative of a highly nonunique solution. Among the infinity of degenerate solutions (most of them badly behaved with arbitrarily large $u(x_\mu)$'s) SVD will select the one of them, call it $\hat{\mathbf{u}}$, with the smallest norm $|\hat{\mathbf{u}}|$ in the sense of

$$\sum_\mu [\hat{u}(x_\mu)]^2 \quad \text{a minimum} \tag{19.4.10}$$

(look at Figure 2.6.1). This solution is often called the *principal solution*. It is a limiting case of what is called *zeroth-order regularization*, corresponding to minimizing the sum of the two positive functionals

$$\hat{\mathbf{u}} \text{ minimizes:} \quad \chi^2[\mathbf{u}] + \lambda(\mathbf{u}\cdot\mathbf{u}) \tag{19.4.11}$$

in the limit of small λ. Below, we will learn how to do such minimizations, as well as more general ones, without the ad hoc use of SVD.

What happens if we determine $\hat{\mathbf{u}}$ by equation (19.4.11) with a non-infinitesimal value of λ? First, note that if $M \gg N$ (many more unknowns than equations), then $\mathbf{u}$ will often have enough freedom to be able to make χ^2 (equation 19.4.9) quite unrealistically small, if not zero. In the language of §15.1, the number of degrees of freedom $\nu = N - M$, which is approximately the expected value of χ^2 when ν is large, is being driven down to zero (and, not meaningfully, beyond). Yet, we know that for the *true* underlying function $u(x)$, which has no adjustable parameters, the number of degrees of freedom and the expected value of χ^2 should be about $\nu \approx N$.

Increasing λ pulls the solution away from minimizing χ^2 in favor of minimizing $\hat{\mathbf{u}}\cdot\hat{\mathbf{u}}$. From the preliminary discussion above, we can view this as minimizing $\hat{\mathbf{u}}\cdot\hat{\mathbf{u}}$ subject to the *constraint* that χ^2 have some constant nonzero value. A popular choice, in fact, is to find that value of λ which yields $\chi^2 = N$, that is, to get about as much extra regularization as a plausible value of χ^2 dictates. The resulting $\hat{u}(x)$ is called *the solution of the inverse problem with zeroth-order regularization*.

The value N is actually a surrogate for any value drawn from a Gaussian distribution with mean N and standard deviation $(2N)^{1/2}$ (the asymptotic χ^2 distribution). One might equally plausibly try two values of λ, one giving $\chi^2 = N + (2N)^{1/2}$, the other $N - (2N)^{1/2}$.

Zeroth-order regularization, though dominated by better methods, demonstrates most of the basic ideas that are used in inverse problem theory. In general, there are two positive functionals, call them $\mathcal{A}$ and $\mathcal{B}$. The first, $\mathcal{A}$, measures something like the agreement of a model to the data (e.g., χ^2), or sometimes a related quantity like the "sharpness" of the mapping between the solution and the underlying function. When $\mathcal{A}$ by itself is minimized, the agreement or sharpness becomes very good (often impossibly good), but the solution becomes unstable, wildly oscillating, or in other ways unrealistic, reflecting that $\mathcal{A}$ alone typically defines a highly degenerate minimization problem.

That is where $\mathcal{B}$ comes in. It measures something like the "smoothness" of the desired solution, or sometimes a related quantity that parametrizes the stability of the solution with respect to variations in the data, or sometimes a quantity reflecting a priori judgments about the likelihood of a solution. $\mathcal{B}$ is called the *stabilizing functional* or *regularizing operator*. In any case, minimizing $\mathcal{B}$ by itself is supposed to give a solution that is "smooth" or "stable" or "likely" — and that has nothing at all to do with the measured data.

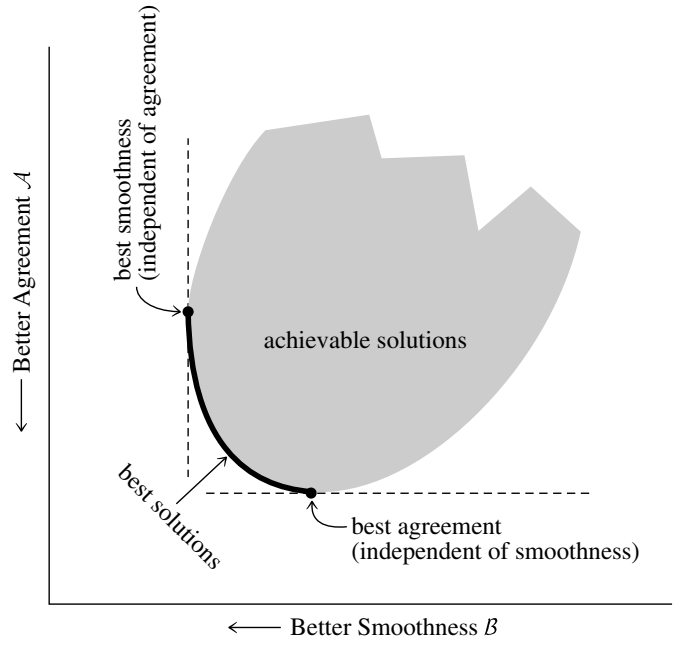


Figure 19.4.1. Almost all inverse problem methods involve a trade-off between two optimizations: agreement between data and solution, or "sharpness" of mapping between true and estimated solutions (here denoted $\mathcal{A}$), and smoothness or stability of the solution (here denoted $\mathcal{B}$). Among all possible solutions, shown here schematically as the shaded region, those on the boundary connecting the unconstrained minimum of $\mathcal{A}$ and the unconstrained minimum of $\mathcal{B}$ are the "best" solutions, in the sense that every other solution is dominated by at least one solution on the curve.

The single central idea in inverse theory is the prescription

$$\text{minimize:} \quad \mathcal{A} + \lambda \mathcal{B} \tag{19.4.12}$$

for various values of $0 < \lambda < \infty$ along the so-called trade-off curve (see Figure 19.4.1), and then to settle on a "best" value of λ by one or another criterion, ranging from fairly objective (e.g., making $\chi^2 = N$) to entirely subjective. Successful methods, several of which we will now describe, differ as to their choices of $\mathcal{A}$ and $\mathcal{B}$, as to whether the prescription (19.4.12) yields linear or nonlinear equations, as to their recommended method for selecting a final λ, and as to their practicality for computer-intensive two-dimensional problems like image processing.

Equation (19.4.12) has a natural Bayesian interpretation that gives some additional insight. Given the data points $\mathbf{c}$ and measurements $\mathbf{u}$, we can use Bayes' law, equation (15.0.1), to write

$$P(\mathbf{u}|\mathbf{c}, I) \propto P(\mathbf{c}|\mathbf{u}, I) P(\mathbf{u}|I) \tag{19.4.13}$$

where $P(\mathbf{u}|I)$ is the Bayesian prior on $\mathbf{u}$ before we see any data, given any background information I. Often, we can usefully write the right-hand side as the product of two exponentials, that is,

$$P(\mathbf{c}|\mathbf{u}, I) \equiv e^{-\mathcal{A}(\mathbf{c},\mathbf{u})}, \qquad P(\mathbf{u}|I) \equiv e^{-\lambda \mathcal{B}(\mathbf{u})} \tag{19.4.14}$$

For example, if the measurement errors are distributed as a multivariate Gaussian, then equation (19.4.9) implies

$$\mathcal{A} = \tfrac{1}{2}\chi^2(\mathbf{c}, \mathbf{u}) \tag{19.4.15}$$

while a Bayesian prior expressing the belief that $\mathbf{u}$ should not have wild, large-amplitude oscillations might be captured by the multivariate Gaussian prior

$$\mathcal{B} = \lambda(\mathbf{u} \cdot \mathbf{u}) \tag{19.4.16}$$

Maximizing $P(\mathbf{u}|\mathbf{c}, I)$ to find the most probable $\hat{\mathbf{u}}$ is now exactly equivalent to equation (19.4.11). The constant λ is now merely a convenient parameterization for the narrowness of the Gaussian prior. It acts as a trade-off parameter, exactly as described above. In subsequent sections we will learn how to devise more sophisticated *smoothness priors*. In several cases these will be positive-definite quadratic forms in $\mathbf{u}$; those cases correspond to more complicated multivariate Gaussian priors on $\mathbf{u}$. In §19.7 we will meet a prior that is not exactly Gaussian, but rather based on the concept of *entropy*.

Within the Bayesian framework, you can do more than just solve for the most likely model $\hat{\mathbf{u}}$. For example, you can use Markov chain Monte Carlo (§15.8) to sample from the distribution of $\mathbf{u}$'s given the observed data.

CITED REFERENCES AND FURTHER READING:

Craig, I.J.D., and Brown, J.C. 1986, *Inverse Problems in Astronomy* (Bristol, UK: Adam Hilger).

Twomey, S. 1977, *Introduction to the Mathematics of Inversion in Remote Sensing and Indirect Measurements* (Amsterdam: Elsevier).

Tikhonov, A.N., and Arsenin, V.Y. 1977, *Solutions of Ill-Posed Problems* (New York: Wiley).

Tikhonov, A.N., and Goncharsky, A.V. (eds.) 1987, *Ill-Posed Problems in the Natural Sciences* (Moscow: MIR).

Parker, R.L. 1977, "Understanding Inverse Theory," *Annual Review of Earth and Planetary Science*, vol. 5, pp. 35–64.

Frieden, B.R. 1975, in *Picture Processing and Digital Filtering*, T.S. Huang, ed. (New York: Springer).

Tarantola, A. 1995, *Inverse Problem Theory and Methods for Model Parameter Estimation* (Philadelphia: S.I.A.M.). Also available at `http://www.ipgp.jussieu.fr/~tarantola/Files/Professional/SIAM`.

Baumeister, J. 1987, *Stable Solution of Inverse Problems* (Braunschweig, Germany: Friedr. Vieweg) [mathematically oriented].

Titterington, D.M. 1985, "General Structure of Regularization Procedures in Image Reconstruction," *Astronomy and Astrophysics*, vol. 144, pp. 381–387.

Jeffrey, W., and Rosner, R. 1986, "On Strategies for Inverting Remote Sensing Data," *Astrophysical Journal*, vol. 310, pp. 463–472.

19.5 Linear Regularization Methods

What we will call *linear regularization* is also called the *Phillips-Twomey method* [1,2], the *constrained linear inversion method* [3], the *method of regularization* [4],

and *Tikhonov-Miller regularization* [5-7]. (It probably has other names also, since it is so obviously a good idea.) In its simplest form, the method is an immediate generalization of zeroth-order regularization (equation 19.4.11, above). As before, the functional $\mathcal{A}$ is taken to be the χ^2 deviation, equation (19.4.9), but the functional $\mathcal{B}$ is replaced by more sophisticated measures of smoothness that derive from first or higher derivatives.

For example, suppose that your a priori belief is that a credible $u(x)$ is not too different from a constant. Then a reasonable functional to minimize is

$$\mathcal{B} \propto \int [\hat{u}'(x)]^2 dx \propto \sum_{\mu=0}^{M-2} [\hat{u}_\mu - \hat{u}_{\mu+1}]^2 \tag{19.5.1}$$

since it is nonnegative and equal to zero only when $\hat{u}(x)$ is constant. Here $\hat{u}_\mu \equiv \hat{u}(x_\mu)$, and the second equality (proportionality) assumes that the x_μ's are uniformly spaced. We can write the second form of $\mathcal{B}$ as

$$\mathcal{B} = |\mathbf{B} \cdot \hat{\mathbf{u}}|^2 = \hat{\mathbf{u}} \cdot (\mathbf{B}^T \cdot \mathbf{B}) \cdot \hat{\mathbf{u}} \equiv \hat{\mathbf{u}} \cdot \mathbf{H} \cdot \hat{\mathbf{u}} \tag{19.5.2}$$

where $\hat{\mathbf{u}}$ is the vector of components $\hat{u}_\mu$, $\mu = 0, \ldots, M-1$; $\mathbf{B}$ is the $(M-1) \times M$ first difference matrix

$$\mathbf{B} = \begin{pmatrix} -1 & 1 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & -1 & 1 & 0 & 0 & 0 & 0 & \cdots & 0 \\ \vdots & & & & \ddots & & & & \vdots \\ 0 & \cdots & 0 & 0 & 0 & 0 & -1 & 1 & 0 \\ 0 & \cdots & 0 & 0 & 0 & 0 & 0 & -1 & 1 \end{pmatrix} \tag{19.5.3}$$

and $\mathbf{H}$ is the $M \times M$ matrix

$$\mathbf{H} = \mathbf{B}^T \cdot \mathbf{B} = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 & 0 & \cdots & 0 \\ \vdots & & & & \ddots & & & & \vdots \\ 0 & \cdots & 0 & 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & \cdots & 0 & 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & \cdots & 0 & 0 & 0 & 0 & 0 & -1 & 1 \end{pmatrix} \tag{19.5.4}$$

Note that $\mathbf{B}$ has one fewer row than column. It follows that the symmetric $\mathbf{H}$ is degenerate; it has exactly one zero eigenvalue corresponding to the *value* of a constant function, any one of which makes $\mathcal{B}$ exactly zero.

If, just as in §15.4, we write

$$A_{i\mu} \equiv R_{i\mu}/\sigma_i \qquad b_i \equiv c_i/\sigma_i \tag{19.5.5}$$

then, using equation (19.4.9), the minimization principle (19.4.12) is

$$\text{minimize:} \quad \mathcal{A} + \lambda\mathcal{B} = |\mathbf{A} \cdot \hat{\mathbf{u}} - \mathbf{b}|^2 + \lambda \hat{\mathbf{u}} \cdot \mathbf{H} \cdot \hat{\mathbf{u}} \tag{19.5.6}$$

This can readily be reduced to a linear set of *normal equations*, just as in §15.4: The components $\hat{u}_\mu$ of the solution satisfy the set of M equations in M unknowns,

$$\sum_\rho \left[\left(\sum_i A_{i\mu} A_{i\rho} \right) + \lambda H_{\mu\rho} \right] \hat{u}_\rho = \sum_i A_{i\mu} b_i \qquad \mu = 0, 1, \ldots, M-1 \tag{19.5.7}$$

or, in vector notation,

$$(\mathbf{A}^T \cdot \mathbf{A} + \lambda \mathbf{H}) \cdot \hat{\mathbf{u}} = \mathbf{A}^T \cdot \mathbf{b} \tag{19.5.8}$$

Equations (19.5.7) or (19.5.8) can be solved by the standard techniques of Chapter 2, e.g., LU decomposition. The usual warnings about normal equations being ill-conditioned do not apply, since the whole purpose of the λ term is to cure that same ill-conditioning. Note, however, that the λ term *by itself* is ill-conditioned, since it does not select a preferred constant value. You hope your data can at least do *that*!

Although inversion of the matrix $(\mathbf{A}^T \cdot \mathbf{A} + \lambda \mathbf{H})$ is not generally the best way to solve for $\hat{\mathbf{u}}$, let us digress to write the solution to equation (19.5.8) schematically as

$$\hat{\mathbf{u}} = \left(\frac{1}{\mathbf{A}^T \cdot \mathbf{A} + \lambda \mathbf{H}} \cdot \mathbf{A}^T \cdot \mathbf{A} \right) \mathbf{A}^{-1} \cdot \mathbf{b} \qquad \text{(schematic only!)} \tag{19.5.9}$$

where the identity matrix in the form $\mathbf{A} \cdot \mathbf{A}^{-1}$ has been inserted. This is schematic not only because the matrix inverse is fancifully written as a denominator, but also because, in general, the inverse matrix $\mathbf{A}^{-1}$ does not exist. However, it is illuminating to compare equation (19.5.9) with equation (13.3.6) for optimal or Wiener filtering, or with equation (13.6.6) for general linear prediction. (The concepts of §15.9 are also related.) One sees that $\mathbf{A}^T \cdot \mathbf{A}$ plays the role of S^2, the signal power or autocorrelation, while $\lambda \mathbf{H}$ plays the role of N^2, the noise power or autocorrelation. The term in parentheses in equation (19.5.9) is something like an optimal filter, whose effect is to pass the ill-posed inverse $\mathbf{A}^{-1} \cdot \mathbf{b}$ through unmodified when $\mathbf{A}^T \cdot \mathbf{A}$ is sufficiently large, but to suppress it when $\mathbf{A}^T \cdot \mathbf{A}$ is small.

The above choices of $\mathbf{B}$ and $\mathbf{H}$ are only the simplest in an obvious sequence of derivatives. If your a priori belief is that a *linear* function is a good approximation to $u(x)$, then minimize

$$\mathcal{B} \propto \int [\hat{u}''(x)]^2 dx \propto \sum_{\mu=0}^{M-3} [-\hat{u}_\mu + 2\hat{u}_{\mu+1} - \hat{u}_{\mu+2}]^2 \tag{19.5.10}$$

implying

$$\mathbf{B} = \begin{pmatrix} -1 & 2 & -1 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 & 0 & \cdots & 0 \\ \vdots & & & & \ddots & & & & \vdots \\ 0 & \cdots & 0 & 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & \cdots & 0 & 0 & 0 & 0 & -1 & 2 & -1 \end{pmatrix} \tag{19.5.11}$$

and

$$\mathbf{H} = \mathbf{B}^T \cdot \mathbf{B} = \begin{pmatrix} 1 & -2 & 1 & 0 & 0 & 0 & 0 & \cdots & 0 \\ -2 & 5 & -4 & 1 & 0 & 0 & 0 & \cdots & 0 \\ 1 & -4 & 6 & -4 & 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & -4 & 6 & -4 & 1 & 0 & \cdots & 0 \\ \vdots & & & & \ddots & & & & \vdots \\ 0 & \cdots & 0 & 1 & -4 & 6 & -4 & 1 & 0 \\ 0 & \cdots & 0 & 0 & 1 & -4 & 6 & -4 & 1 \\ 0 & \cdots & 0 & 0 & 0 & 1 & -4 & 5 & -2 \\ 0 & \cdots & 0 & 0 & 0 & 0 & 1 & -2 & 1 \end{pmatrix} \tag{19.5.12}$$

This $\mathbf{H}$ has two zero eigenvalues, corresponding to the two undetermined parameters of a linear function.

If your a priori belief is that a *quadratic* function is preferable, then minimize

$$\mathcal{B} \propto \int [\hat{u}'''(x)]^2 dx \propto \sum_{\mu=0}^{M-4} [-\hat{u}_\mu + 3\hat{u}_{\mu+1} - 3\hat{u}_{\mu+2} + \hat{u}_{\mu+3}]^2 \tag{19.5.13}$$

with

$$\mathbf{B} = \begin{pmatrix} -1 & 3 & -3 & 1 & 0 & 0 & 0 & \cdots & 0 \\ 0 & -1 & 3 & -3 & 1 & 0 & 0 & \cdots & 0 \\ \vdots & & & & \ddots & & & & \vdots \\ 0 & \cdots & 0 & 0 & -1 & 3 & -3 & 1 & 0 \\ 0 & \cdots & 0 & 0 & 0 & -1 & 3 & -3 & 1 \end{pmatrix} \tag{19.5.14}$$

and now

$$\mathbf{H} = \begin{pmatrix} 1 & -3 & 3 & -1 & 0 & 0 & 0 & 0 & 0 & \cdots & 0 \\ -3 & 10 & -12 & 6 & -1 & 0 & 0 & 0 & 0 & \cdots & 0 \\ 3 & -12 & 19 & -15 & 6 & -1 & 0 & 0 & 0 & \cdots & 0 \\ -1 & 6 & -15 & 20 & -15 & 6 & -1 & 0 & 0 & \cdots & 0 \\ 0 & -1 & 6 & -15 & 20 & -15 & 6 & -1 & 0 & \cdots & 0 \\ \vdots & & & & & \ddots & & & & & \vdots \\ 0 & \cdots & 0 & -1 & 6 & -15 & 20 & -15 & 6 & -1 & 0 \\ 0 & \cdots & 0 & 0 & -1 & 6 & -15 & 20 & -15 & 6 & -1 \\ 0 & \cdots & 0 & 0 & 0 & -1 & 6 & -15 & 19 & -12 & 3 \\ 0 & \cdots & 0 & 0 & 0 & 0 & -1 & 6 & -12 & 10 & -3 \\ 0 & \cdots & 0 & 0 & 0 & 0 & 0 & -1 & 3 & -3 & 1 \end{pmatrix} \tag{19.5.15}$$

(We'll leave the calculation of cubics and above to the compulsive reader.)

Notice that you can regularize with "closeness to a differential equation," if you want. Just pick $\mathbf{B}$ to be the appropriate sum of finite difference operators (the coefficients can depend on x) and calculate $\mathbf{H} = \mathbf{B}^T \cdot \mathbf{B}$. You don't need to know the values of your boundary conditions, since $\mathbf{B}$ can have fewer rows than columns, as above; hopefully, your data will determine them. Of course, if you do know some boundary conditions, you can build these into $\mathbf{B}$, too.

With all the proportionality signs above, you may have lost track of what actual value of λ to try first. A simple trick for at least getting "on the map" is to first try

$$\lambda = \mathrm{Tr}(\mathbf{A}^T \cdot \mathbf{A})/\mathrm{Tr}(\mathbf{H}) \tag{19.5.16}$$

where Tr is the trace of the matrix (sum of diagonal components). This choice will tend to make the two parts of the minimization have comparable weights, and you can adjust from there.

As for what is the "correct" value of λ, an objective criterion, if you know your errors σ_i with reasonable accuracy, is to make χ^2 (that is, $|\mathbf{A} \cdot \hat{\mathbf{u}} - \mathbf{b}|^2$) equal to N, the number of measurements. We remarked above on the twin acceptable

choices $N \pm (2N)^{1/2}$. A subjective criterion is to pick any value that you like in the range $0 < \lambda < \infty$, depending on your relative degree of belief in the a priori and a posteriori evidence. (Yes, people actually do that.) The problem with being a rigorous Bayesian at this stage is that rarely, if ever, is your understanding of the prior so complete as to give a firm, objective, value for λ, and, as we pointed out in §15.1.1, purely Bayesian methods for assessing goodness-of-fit are largely nonexistent.

19.5.1 Two-Dimensional Problems and Iterative Methods

Up to now our notation has been indicative of a one-dimensional problem, finding $\hat{u}(x)$ or $\hat{u}_\mu = \hat{u}(x_\mu)$. However, all of the discussion easily generalizes to the problem of estimating a two-dimensional set of unknowns $\hat{u}_{\mu\kappa}$, $\mu = 0, \ldots, M-1$, $\kappa = 0, \ldots, K-1$, corresponding, say, to the pixel intensities of a measured image. In this case, equation (19.5.8) is still the one we want to solve.

In image processing, it is usual to have the same number of input pixels in a measured "raw" or "dirty" image as desired "clean" pixels in the processed output image, so the matrices $\mathbf{R}$ and $\mathbf{A}$ (equation 19.5.5) are square and of size $MK \times MK$. $\mathbf{A}$ is typically too large to represent as a full matrix, but often it is either (i) sparse, with coefficients blurring an underlying pixel (i, j) only into measurements $(i \pm$ few, $j \pm$ few), or (ii) translationally invariant, so that $A_{(i,j)(\mu,\nu)} = A(i-\mu, j-\nu)$. Both of these situations lead to tractable problems.

In the case of translational invariance, fast Fourier transforms (FFTs) are the obvious method of choice. The general linear relation between underlying function and measured values (19.4.7) now becomes a discrete convolution like equation (13.1.1). If $\mathbf{k}$ denotes a two-dimensional wave vector, then the two-dimensional FFT takes us back and forth between the transform pairs

$$A(i-\mu, j-\nu) \iff \tilde{\mathbf{A}}(\mathbf{k}) \qquad b_{(i,j)} \iff \tilde{b}(\mathbf{k}) \qquad \hat{u}_{(i,j)} \iff \tilde{u}(\mathbf{k}) \tag{19.5.17}$$

We also need a regularization or smoothing operator $\mathbf{B}$ and the derived $\mathbf{H} = \mathbf{B}^T \cdot \mathbf{B}$. One popular choice for $\mathbf{B}$ is the five-point finite difference approximation of the Laplacian operator, that is, the difference between the value of each point and the average of its four Cartesian neighbors. In Fourier space, this choice implies

$$\begin{aligned} \tilde{B}(\mathbf{k}) &\propto \sin^2(\pi k_1/M)\sin^2(\pi k_2/K) \\ \tilde{H}(\mathbf{k}) &\propto \sin^4(\pi k_1/M)\sin^4(\pi k_2/K) \end{aligned} \tag{19.5.18}$$

In Fourier space, equation (19.5.7) is merely algebraic, with solution

$$\tilde{u}(\mathbf{k}) = \frac{\tilde{A}^*(\mathbf{k})\tilde{b}(\mathbf{k})}{|\tilde{A}(\mathbf{k})|^2 + \lambda \tilde{H}(\mathbf{k})} \tag{19.5.19}$$

where the asterisk denotes complex conjugation. You can make use of the FFT routines for real data in §12.6.

Turn now to the case where $\mathbf{A}$ is not translationally invariant. Direct solution of (19.5.8) is now hopeless, since the matrix $\mathbf{A}$ is just too large. We need some kind of iterative scheme.

One way to proceed is to use the full machinery of the conjugate gradient method in §10.8 to find the minimum of $\mathcal{A} + \lambda\mathcal{B}$, equation (19.5.6). Of the various methods in Chapter 10, the conjugate gradient is the unique best choice because (i) it does not require storage of a Hessian matrix, which would be infeasible here, and (ii) it does exploit gradient information, which we can readily compute: The gradient of equation (19.5.6) is

$$\nabla(\mathcal{A} + \lambda\mathcal{B}) = 2[(\mathbf{A}^T \cdot \mathbf{A} + \lambda\mathbf{H}) \cdot \hat{\mathbf{u}} - \mathbf{A}^T \cdot \mathbf{b}] \tag{19.5.20}$$

(cf. 19.5.8). Evaluation of both the function and the gradient should of course take advantage of the sparsity of $\mathbf{A}$, for example using the methods `ax` and `atx` in the `NRsparseMat` object in §2.7.5. We will discuss the conjugate gradient technique further in §19.7, in the context of the (nonlinear) maximum entropy method. Some of that discussion can apply here as well.

The conjugate gradient method notwithstanding, application of the unsophisticated steepest descent method (see §10.8) can sometimes produce useful results, particularly when combined with projections onto convex sets (see below). If the solution after k iterations is denoted $\hat{\mathbf{u}}^{(k)}$, then after $k + 1$ iterations we have

$$\hat{\mathbf{u}}^{(k+1)} = [\mathbf{1} - \epsilon(\mathbf{A}^T \cdot \mathbf{A} + \lambda\mathbf{H})] \cdot \hat{\mathbf{u}}^{(k)} + \epsilon\mathbf{A}^T \cdot \mathbf{b} \tag{19.5.21}$$

Here ϵ is a parameter that dictates how far to move in the downhill gradient direction. The method converges when ϵ is small enough, in particular satisfying

$$0 < \epsilon < \frac{2}{\text{max eigenvalue } (\mathbf{A}^T \cdot \mathbf{A} + \lambda\mathbf{H})} \tag{19.5.22}$$

There exist complicated schemes for finding optimal values or sequences for ϵ, see [7]; or, one can adopt an experimental approach, evaluating (19.5.6) to be sure that downhill steps are in fact being taken.

In those image processing problems where the final measure of success is somewhat subjective (e.g., "how good does the picture look?"), iteration (19.5.21) sometimes produces significantly improved images long before convergence is achieved. This probably accounts for much of its use, since its mathematical convergence is extremely slow. In fact, (19.5.21) can be used with $\mathbf{H} = 0$, in which case the solution is not regularized at all, and full convergence would be disastrous! This is called *Van Cittert's method* and goes back to the 1930s. A number of iterations the order of 10^3 is not uncommon [7].

19.5.2 Deterministic Constraints: Projections onto Convex Sets

A set of possible underlying functions (or images) $\{\hat{\mathbf{u}}\}$ is said to be *convex* if, for any two elements $\hat{\mathbf{u}}_a$ and $\hat{\mathbf{u}}_b$ in the set, all the linearly interpolated combinations

$$(1 - \eta)\hat{\mathbf{u}}_a + \eta\hat{\mathbf{u}}_b \qquad 0 \le \eta \le 1 \tag{19.5.23}$$

are also in the set. Many *deterministic constraints* that one might want to impose on the solution $\hat{\mathbf{u}}$ to an inverse problem in fact define convex sets, for example:

- positivity
- compact support (i.e., zero value outside of a certain region)
- known bounds (i.e., $u_L(x) \le \hat{u}(x) \le u_U(x)$ for specified functions u_L and u_U).

(In this last case, the bounds might be related to an initial estimate and its error bars, e.g., $\hat{u}_0(x) \pm \gamma\sigma(x)$, where γ is of order 1 or 2.) Notice that these, and similar, constraints can be either in the image space, or in the Fourier transform space, or (in fact) in the space of any linear transformation of $\hat{\mathbf{u}}$.

If C_i is a convex set, then $\mathcal{P}_i$ is called a *nonexpansive projection operator* onto that set if (i) $\mathcal{P}_i$ leaves unchanged any $\hat{\mathbf{u}}$ already in C_i, and (ii) $\mathcal{P}_i$ maps any $\hat{\mathbf{u}}$ outside C_i to the *closest* element of C_i, in the sense that

$$|\mathcal{P}_i\hat{\mathbf{u}} - \hat{\mathbf{u}}| \le |\hat{\mathbf{u}}_a - \hat{\mathbf{u}}| \quad \text{for all } \hat{\mathbf{u}}_a \text{ in } C_i \tag{19.5.24}$$

While this definition sounds complicated, examples are very simple: A nonexpansive projection onto the set of positive $\hat{\mathbf{u}}$'s is "set all negative components of $\hat{\mathbf{u}}$ equal to zero." A nonexpansive projection onto the set of $\hat{u}(x)$'s bounded by $u_L(x) \le \hat{u}(x) \le u_U(x)$ is "set all values less than the lower bound equal to that bound, and set all values greater than the upper bound equal to *that* bound." A nonexpansive projection onto functions with compact support is "zero the values outside of the region of support."

The usefulness of these definitions is the following remarkable theorem: Let C be the intersection of m convex sets $C_1, C_2, \ldots, C_m$. Then the iteration

$$\hat{\mathbf{u}}^{(k+1)} = (\mathcal{P}_1\mathcal{P}_2\cdots\mathcal{P}_m)\hat{\mathbf{u}}^{(k)} \tag{19.5.25}$$

will converge to C from all starting points, as $k \to \infty$. Also, if C is empty (there is no intersection), then the iteration will have no limit point. Application of this theorem is called the *method of projections onto convex sets* or sometimes *POCS* [7].

A generalization of the POCS theorem is that the $\mathcal{P}_i$'s can be replaced by a set of $\mathcal{T}_i$'s,

$$\mathcal{T}_i \equiv \mathbf{1} + \beta_i(\mathcal{P}_i - \mathbf{1}) \qquad 0 < \beta_i < 2 \tag{19.5.26}$$

A well-chosen set of β_i's can accelerate the convergence to the intersection set C.

Some inverse problems can be completely solved by iteration (19.5.25) alone! For example, a problem that occurs in both astronomical imaging and X-ray diffraction work is to recover an image given only the *modulus* of its Fourier transform (equivalent to its power spectrum or autocorrelation) and not the *phase*. Here two convex sets can be utilized: the set of all positive images, and the set of all images with zero intensity outside of some specified region. A third set, the set of all images whose Fourier transform has the specified modulus to within specified error bounds, is not convex: It is an annulus in the complex plane for each Fourier component. (Of course FFTs are used to get in and out of Fourier space each time the Fourier constraint is imposed.) The POCS iteration (19.5.25) that cycles among all three sets, imposing each constraint in turn, is thus not guaranteed (by the POCS theorem) to converge; it can get stuck in *traps* [8]. However, it often does work.

The specific application of POCS to constraints (not necessarily all convex) alternately in the spatial and Fourier domains is known as the *Gerchberg-Saxton* algorithm [9]. While this algorithm is nonexpansive, and is frequently convergent

in practice, it does not converge in all cases [8,10]. In the phase-retrieval problem just mentioned, the algorithm often gets stuck in traps for many iterations. After as many as 10^4 to 10^5 iterations, sudden, dramatic improvements may occur. In principle, some traps may be permanent, requiring more interventional "unsticking" procedures; see [8,11]. The uniqueness of the solution is also not well understood, although for two-dimensional images of reasonable complexity it is believed to be unique. The use of nonconvex sets in an iteration like (19.5.25) is called the *method of generalized projections*.

Deterministic constraints can be incorporated, via projection operators, into iterative methods of linear regularization. In particular, rearranging terms somewhat, we can write the iteration (19.5.21) as

$$\hat{\mathbf{u}}^{(k+1)} = (\mathbf{1} - \epsilon\lambda\mathbf{H}) \cdot \hat{\mathbf{u}}^{(k)} + \epsilon\mathbf{A}^T \cdot (\mathbf{b} - \mathbf{A} \cdot \hat{\mathbf{u}}^{(k)}) \tag{19.5.27}$$

If the iteration is modified by the insertion of projection operators at each step

$$\hat{\mathbf{u}}^{(k+1)} = (\mathcal{P}_1\mathcal{P}_2\cdots\mathcal{P}_m)[(\mathbf{1} - \epsilon\lambda\mathbf{H}) \cdot \hat{\mathbf{u}}^{(k)} + \epsilon\mathbf{A}^T \cdot (\mathbf{b} - \mathbf{A} \cdot \hat{\mathbf{u}}^{(k)})] \tag{19.5.28}$$

(or, instead of $\mathcal{P}_i$'s, the $\mathcal{T}_i$ operators of equation 19.5.26), then it can be shown that the convergence condition (19.5.22) is unmodified, and the iteration will converge to minimize the quadratic functional (19.5.6) subject to the desired nonlinear deterministic constraints. See [7] for references to more sophisticated, and faster converging, iterations along these lines.

CITED REFERENCES AND FURTHER READING:

Phillips, D.L. 1962, "A Technique for the Numerical Solution of Certain Integral Equations of the First Kind," *Journal of the Association for Computing Machinery*, vol. 9, pp. 84–97.[1]

Twomey, S. 1963, "On the Numerical Solution of Fredholm Integral Equations of the First Kind by the Inversion of the Linear System Produced by Quadrature," *Journal of the Association for Computing Machinery*, vol. 10, pp. 97–101.[2]

Twomey, S. 1977, *Introduction to the Mathematics of Inversion in Remote Sensing and Indirect Measurements* (Amsterdam: Elsevier).[3]

Craig, I.J.D., and Brown, J.C. 1986, *Inverse Problems in Astronomy* (Bristol, UK: Adam Hilger).[4]

Tikhonov, A.N., and Arsenin, V.Y. 1977, *Solutions of Ill-Posed Problems* (New York: Wiley).[5]

Tikhonov, A.N., and Goncharsky, A.V. (eds.) 1987, *Ill-Posed Problems in the Natural Sciences* (Moscow: MIR).

Miller, K. 1970, "Least Squares Methods for Ill-Posed Problems with a Prescribed Bound," *SIAM Journal on Mathematical Analysis*, vol. 1, pp. 52–74.[6]

Schafer, R.W., Mersereau, R.M., and Richards, M.A. 1981, "Constrained Iterative Restoration Algorithm," *Proceedings of the IEEE*, vol. 69, pp. 432–450.

Biemond, J., Lagendijk, R.L., and Mersereau, R.M. 1990, "Iterative Methods for Image Deblurring," *Proceedings of the IEEE*, vol. 78, pp. 856–883.[7]

Sezan, M.I. 1992, "An Overview of Convex Projections Theory and Its Application to Image Recovery Problems," *Ultramicroscopy*, vol. 40, pp. 55–67.[8]

Gerchberg, R.W., and Saxton, W.O. 1972, "A Practical Algorithm for the Determination of Phase from Image and Diffraction Plane Pictures," *Optik*, vol. 35, pp. 237–246.[9]

Fienup, J.R. 1982, "Phase Retrieval Algorithms: A Comparison," *Applied Optics*, vol. 15, pp. 2758–2769.[10]

Fienup, J.R., and Wackerman, C.C. 1986, "Phase-Retrieval Stagnation Problems and Solutions," *Journal of the Optical Society of America A*, vol. 3, pp. 1897–1907.[11]

19.6 Backus-Gilbert Method

The *Backus-Gilbert method* [1,2], also known as the *optimally localized average (OLA)* method (see, e.g., [3] or [4] for summaries) differs from other regularization methods in the nature of its functionals $\mathcal{A}$ and $\mathcal{B}$. For $\mathcal{B}$, the method seeks to maximize the *stability* of the solution $\hat{u}(x)$ rather than, in the first instance, its smoothness. That is,

$$\mathcal{B} \equiv \mathrm{Var}[\hat{u}(x)] \tag{19.6.1}$$

is used as a measure of how much the solution $\hat{u}(x)$ varies as the data vary within their measurement errors. Note that this variance is not the expected deviation of $\hat{u}(x)$ from the true $u(x)$ — that will be constrained by $\mathcal{A}$ — but rather measures the expected experiment-to-experiment scatter among estimates $\hat{u}(x)$ if the whole experiment were to be repeated many times.

For $\mathcal{A}$ the Backus-Gilbert method looks at the relationship between the solution $\hat{u}(x)$ and the true function $u(x)$, and seeks to make the mapping between these as close to the identity map as possible in the limit of error-free data. The method is linear, so the relationship between $\hat{u}(x)$ and $u(x)$ can be written as

$$\hat{u}(x) = \int \hat{\delta}(x, x')u(x')dx' \tag{19.6.2}$$

for some so-called *resolution function* or *averaging kernel* $\hat{\delta}(x, x')$. The Backus-Gilbert method seeks to minimize the width or *spread* of $\hat{\delta}$ (that is, maximize the resolving power). $\mathcal{A}$ is chosen to be some positive measure of the spread.

While Backus-Gilbert's philosophy is thus rather different from that of Phillips-Twomey and related methods, in practice the differences between the methods are less than one might think. A *stable* solution is almost inevitably bound to be *smooth*: The wild, unstable oscillations that result from an unregularized solution are always exquisitely sensitive to small changes in the data. Likewise, making $\hat{u}(x)$ close to $u(x)$ inevitably will bring error-free data into agreement with the model. Thus $\mathcal{A}$ and $\mathcal{B}$ play roles closely analogous to their corresponding roles in the previous two sections.

The principal advantage of the Backus-Gilbert formulation is that it gives good control over just those properties that it seeks to measure, namely stability and resolving power. Moreover, in the Backus-Gilbert method, the choice of λ (playing its usual role of compromise between $\mathcal{A}$ and $\mathcal{B}$) is conventionally made, or at least can easily be made, *before* any actual data are processed. One's uneasiness at making a post hoc, and therefore potentially subjectively biased, choice of λ is thus removed. Backus-Gilbert is often recommended as the method of choice for designing and predicting the performance of experiments that require data inversion.

Let's see how this all works. Starting with equation (19.4.5),

$$c_i \equiv s_i + n_i = \int r_i(x)u(x)dx + n_i \tag{19.6.3}$$

and building in linearity from the start, we seek a set of *inverse response kernels* $q_i(x)$ such that

$$\hat{u}(x) = \sum_i q_i(x)c_i \tag{19.6.4}$$

is the desired estimator of $u(x)$. It is useful to define the integrals of the response kernels for each data point,

$$R_i \equiv \int r_i(x)dx \tag{19.6.5}$$

Substituting equation (19.6.4) into equation (19.6.3), and comparing with equation (19.6.2), we see that

$$\widehat{\delta}(x,x') = \sum_i q_i(x) r_i(x') \tag{19.6.6}$$

We can require this averaging kernel to have unit area at every x, giving

$$1 = \int \widehat{\delta}(x,x')dx' = \sum_i q_i(x) \int r_i(x')dx' = \sum_i q_i(x) R_i \equiv \mathbf{q}(x)\cdot\mathbf{R} \tag{19.6.7}$$

where $\mathbf{q}(x)$ and $\mathbf{R}$ are each vectors of length N, the number of measurements.

Standard propagation of errors, and equation (19.6.1), give

$$\mathcal{B} = \mathrm{Var}[\widehat{u}(x)] = \sum_i \sum_j q_i(x) S_{ij} q_j(x) = \mathbf{q}(x)\cdot\mathbf{S}\cdot\mathbf{q}(x) \tag{19.6.8}$$

where S_{ij} is the covariance matrix (equation 19.4.6). If one can neglect off-diagonal covariances (as when the errors on the c_i's are independent), then $S_{ij} = \delta_{ij}\sigma_i^2$ is diagonal.

We now need to define a measure of the width or spread of $\widehat{\delta}(x,x')$ at each value of x. While many choices are possible, Backus and Gilbert choose the second moment of its square. This measure becomes the functional $\mathcal{A}$,

$$\begin{aligned} \mathcal{A} \equiv w(x) &= \int (x'-x)^2 [\widehat{\delta}(x,x')]^2 dx' \\ &= \sum_i \sum_j q_i(x) W_{ij}(x) q_j(x) \equiv \mathbf{q}(x)\cdot\mathbf{W}(x)\cdot\mathbf{q}(x) \end{aligned} \tag{19.6.9}$$

where we have here used equation (19.6.6) and defined the *spread matrix* $\mathbf{W}(x)$ by

$$W_{ij}(x) \equiv \int (x'-x)^2 r_i(x') r_j(x') dx' \tag{19.6.10}$$

The functions $q_i(x)$ are now determined by the minimization principle

$$\text{minimize:} \quad \mathcal{A} + \lambda\mathcal{B} = \mathbf{q}(x)\cdot\big[\mathbf{W}(x) + \lambda\mathbf{S}\big]\cdot\mathbf{q}(x) \tag{19.6.11}$$

subject to the constraint (19.6.7) that $\mathbf{q}(x)\cdot\mathbf{R} = 1$.

The solution of equation (19.6.11) is

$$\mathbf{q}(x) = \frac{[\mathbf{W}(x) + \lambda\mathbf{S}]^{-1}\cdot\mathbf{R}}{\mathbf{R}\cdot[\mathbf{W}(x) + \lambda\mathbf{S}]^{-1}\cdot\mathbf{R}} \tag{19.6.12}$$

(Reference [4] gives an accessible proof.) For any particular data set $\mathbf{c}$ (set of measurements c_i), the solution $\widehat{u}(x)$ is thus

$$\widehat{u}(x) = \frac{\mathbf{c}\cdot[\mathbf{W}(x) + \lambda\mathbf{S}]^{-1}\cdot\mathbf{R}}{\mathbf{R}\cdot[\mathbf{W}(x) + \lambda\mathbf{S}]^{-1}\cdot\mathbf{R}} \tag{19.6.13}$$

(Don't let this notation mislead you into inverting the full matrix $\mathbf{W}(x) + \lambda\mathbf{S}$. You only need to solve the linear system $(\mathbf{W}(x) + \lambda\mathbf{S}) \cdot \mathbf{y} = \mathbf{R}$ for the vector $\mathbf{y}$, and then substitute $\mathbf{y}$ into both the numerators and denominators of 19.6.12 or 19.6.13.)

Equations (19.6.12) and (19.6.13) have a completely different character from the linearly regularized solutions to (19.5.7) and (19.5.8). The vectors and matrices in (19.6.12) all have size N, the number of measurements. There is no discretization of the underlying variable x, so M does not come into play at all. One solves a different $N \times N$ set of linear equations for each desired value of x. By contrast, in (19.5.8), one solves an $M \times M$ linear set, but only once. In general, the computational burden of repeatedly solving linear systems makes the Backus-Gilbert method unsuitable for other than one-dimensional problems.

How does one choose λ within the Backus-Gilbert scheme? As already mentioned, you can (in some cases *should*) make the choice *before* you see any actual data. For a given trial value of λ, and for a sequence of x's, use equation (19.6.12) to calculate $\mathbf{q}(x)$; then use equation (19.6.6) to plot the resolution functions $\widehat{\delta}(x, x')$ as a function of x'. These plots will exhibit the amplitude with which different underlying values x' contribute to the point $\widehat{u}(x)$ of your estimate. For the same value of λ, also plot the function $\sqrt{\mathrm{Var}[\widehat{u}(x)]}$ using equation (19.6.8). (You need an estimate of your measurement covariance matrix for this.)

As you change λ you will see very explicitly the trade-off between resolution and stability. Pick the value that meets your needs. You can even choose λ to be a function of x, $\lambda = \lambda(x)$, in equations (19.6.12) and (19.6.13), should you desire to do so. (This is one benefit of solving a separate set of equations for each x.) For the chosen value or values of λ, you now have a quantitative understanding of your inverse solution procedure. This can prove invaluable if — once you are processing real data — you need to judge whether a particular feature, a spike or jump for example, is genuine, and/or is actually resolved. The Backus-Gilbert method has found particular success among geophysicists, who use it to obtain information about the structure of the Earth (e.g., density run with depth) from seismic travel time data.

CITED REFERENCES AND FURTHER READING:

Backus, G.E., and Gilbert, F. 1968, "The Resolving Power of Gross Earth Data," *Geophysical Journal of the Royal Astronomical Society*, vol. 16, pp. 169–205.[1]

Backus, G.E., and Gilbert, F. 1970, "Uniqueness in the Inversion of Inaccurate Gross Earth Data," *Philosophical Transactions of the Royal Society of London A*, vol. 266, pp. 123–192.[2]

Parker, R.L. 1977, "Understanding Inverse Theory," *Annual Review of Earth and Planetary Science*, vol. 5, pp. 35–64.[3]

Loredo, T.J., and Epstein, R.I. 1989, "Analyzing Gamma-Ray Burst Spectral Data," *Astrophysical Journal*, vol. 336, pp. 896–919.[4]

19.7 Maximum Entropy Image Restoration

We must first comment in passing that the connection between maximum entropy inverse methods, considered here, and maximum entropy spectral estimation, discussed in §13.7, is rather distant. For practical purposes, the two techniques, though both named *maximum entropy method* or *MEM*, are unrelated. On the other

hand, what we discuss here has a close connection to the discussion of entropy in §14.7.

The entropy of a physical system in some macroscopic state, usually denoted S, is the logarithm of the number of microscopically distinct configurations that all have the same macroscopic observables (i.e., consistent with the observed macroscopic state). Actually, we will find it useful to denote the *negative* of the entropy, also called the *negentropy*, by $H \equiv -S$ (a notation that goes back to Boltzmann). In situations where there is reason to believe that the a priori probabilities of the *microscopic* configurations are all the same (these situations are called *ergodic*), the Bayesian prior $P(\mathbf{u}|I)$ for a *macroscopic* state with entropy S is proportional to $\exp(S)$ or $\exp(-H)$.

MEM uses this concept to assign a prior probability to any given underlying function $\mathbf{u}$. This very general idea is applicable to much more than image restoration [1,2]. For definiteness, however, we consider that application only. Suppose [3-5] that the measurement of luminance in each pixel in an image is quantized to (in some units) an integer value. Let

$$U = \sum_{\mu=0}^{M-1} u_\mu \tag{19.7.1}$$

be the total number of luminance quanta in the whole image. Then we can base our "prior" on the notion that each luminance quantum has an equal a priori chance of being in any pixel. (See [6] for a more abstract justification of this idea.) The number of ways of getting a particular configuration $\mathbf{u}$ is

$$\frac{U!}{u_0!u_1!\cdots u_{M-1}!} \propto \exp\left[-\sum_\mu u_\mu \ln(u_\mu/U) + \tfrac{1}{2}\left(\ln U - \sum_\mu \ln u_\mu\right)\right] \tag{19.7.2}$$

Here the left side can be understood as the number of distinct orderings of all the luminance quanta, divided by the numbers of equivalent reorderings within each pixel, while the right side follows by Stirling's approximation to the factorial function. Taking the negative of the logarithm, and neglecting terms of order $\log U$ in the presence of terms of order U, we get the negentropy

$$H(\mathbf{u}) = \sum_{\mu=0}^{M-1} u_\mu \ln(u_\mu/U) \tag{19.7.3}$$

As discussed for equations (19.4.13) – (19.4.15), we now seek to maximize

$$P(\mathbf{u}|\mathbf{c}, I) \propto \exp\left[-\tfrac{1}{2}\chi^2\right] \exp[-H(\mathbf{u})] \tag{19.7.4}$$

or, equivalently,

$$\text{minimize:} \quad -\ln[\,P(\mathbf{u}|\mathbf{c}, I)\,] = \tfrac{1}{2}\chi^2[\mathbf{u}] + H(\mathbf{u}) = \tfrac{1}{2}\chi^2[\mathbf{u}] + \sum_{\mu=0}^{M-1} u_\mu \ln(u_\mu/U) \tag{19.7.5}$$

This ought to remind you of equation (19.4.11), or equation (19.5.6), or in fact any of our previous minimization principles along the lines of $\mathcal{A}+\lambda\mathcal{B}$, where $\lambda\mathcal{B} = H(\mathbf{u})$ is a regularizing operator. Where is λ? We need to put it in for exactly the reason discussed following equation (19.4.11): Degenerate inversions are likely to be able to achieve unrealistically small values of χ^2. We need an adjustable parameter to bring χ^2 into its expected narrow statistical range of $N \pm (2N)^{1/2}$. The discussion at the beginning of §19.4 showed that it makes no difference which term we attach the λ to. For consistency in notation, we absorb a factor 2 into λ and put it on the entropy term. (Another way to see the necessity of an undetermined λ factor is to note that it is necessary if our minimization principle is to be invariant under changing the units in which $\mathbf{u}$ is quantized, e.g., if an 8-bit analog-to-digital converter is replaced by a 12-bit one.) We can now also put "hats" back to indicate that this is the procedure for obtaining our chosen statistical estimator:

$$\hat{\mathbf{u}} \text{ minimizes:} \quad \mathcal{A}+\lambda\mathcal{B} = \chi^2[\mathbf{u}] + \lambda H(\mathbf{u}) = \chi^2[\mathbf{u}] + \lambda \sum_{\mu=0}^{M-1} u_\mu \ln(u_\mu) \tag{19.7.6}$$

(Formally, we might also add a second Lagrange multiplier $\lambda' U$, to constrain the total intensity U to be constant.)

It is not hard to see that the negentropy, $H(\mathbf{u})$, is in fact a regularizing operator, similar to $\mathbf{u}\cdot\mathbf{u}$ (equation 19.4.11) or $\mathbf{u}\cdot\mathbf{H}\cdot\mathbf{u}$ (equation 19.5.6). The following of its properties are noteworthy:

1. When U is held constant, $H(\mathbf{u})$ is minimized for $\hat{u}_\mu = U/M = \text{constant}$, so it smooths in the sense of trying to achieve a constant solution, similar to equation (19.5.4). The fact that the constant solution is a minimum follows from the fact that the second derivative of $u \ln u$ is positive.
2. Unlike equation (19.5.4), however, $H(\hat{\mathbf{u}})$ is *local*, in the sense that it does not difference neighboring pixels. It simply sums some function f, here
$$f(u) = u \ln u \tag{19.7.7}$$
over all pixels; it is invariant, in fact, under a complete scrambling of the pixels in an image. This form implies that $H(\mathbf{u})$ is not seriously increased by the occurrence of a small number of very bright pixels (point sources) embedded in a low-intensity smooth background.
3. $H(\mathbf{u})$ goes to infinite slope as any one pixel goes to zero. This causes it to enforce positivity of the image, without the necessity of additional deterministic constraints.
4. The biggest difference between $H(\mathbf{u})$ and the other regularizing operators that we have met is that $H(\mathbf{u})$ is not a quadratic functional of $\mathbf{u}$, so the equations obtained by varying equation (19.7.6) are *nonlinear*. This fact is itself worthy of some additional discussion.

Nonlinear equations are harder to solve than linear equations. For image processing, however, the large number of equations usually dictates an iterative solution procedure, even for linear equations, so the practical effect of the nonlinearity is somewhat mitigated. Below, we will summarize some of the methods that are successfully used for MEM inverse problems.

For some problems, notably the problem in radio-astronomy of image recovery from an incomplete set of Fourier coefficients, the superior performance of MEM

inversion can be, in part, traced to the nonlinearity of $H(\mathbf{u})$. One way to see this [3] is to consider the limit of perfect measurements $\sigma_i \to 0$. In this case, the χ^2 term in the minimization principle (19.7.6) gets replaced by a set of constraints, each with its own Lagrange multiplier, requiring agreement between model and data; that is,

$$\hat{\mathbf{u}} \text{ minimizes:} \quad \sum_j \lambda_j \left[c_j - \sum_\mu R_{j\mu} u_\mu \right] + H(\mathbf{u}) \tag{19.7.8}$$

(cf. equation 19.4.7). Setting the formal derivative with respect to u_μ to zero gives

$$\frac{\partial H}{\partial u_\mu} = f'(u_\mu) = \sum_j \lambda_j R_{j\mu} \tag{19.7.9}$$

or defining a function G as the inverse function of f',

$$u_\mu = G\left(\sum_j \lambda_j R_{j\mu} \right) \tag{19.7.10}$$

This solution is only formal, since the λ_j's must be found by requiring that equation (19.7.10) satisfy all the constraints built into equation (19.7.8). However, equation (19.7.10) does show the crucial fact that if G is *linear*, then the solution $\hat{\mathbf{u}}$ contains *only* a linear combination of basis functions $R_{j\mu}$ corresponding to actual measurements j. This is equivalent to setting unmeasured c_j's to zero. Notice that the principal solution obtained from equation (19.4.11) in fact has a linear G.

In the problem of incomplete Fourier image reconstruction, the typical $R_{j\mu}$ has the form $\exp(-2\pi i \mathbf{k}_j \cdot \mathbf{x}_\mu)$, where $\mathbf{x}_\mu$ is a two-dimensional vector in the image space and $\mathbf{k}_\mu$ is a two-dimensional wave vector. If an image contains strong point sources, then the effect of setting unmeasured c_j's to zero is to produce sidelobe ripples throughout the image plane. These ripples can mask any actual extended, low-intensity image features lying between the point sources. If, however, the slope of G is smaller for small values of its argument and larger for large values, then ripples in low-intensity portions of the image are relatively suppressed, while strong point sources will be relatively sharpened ("superresolution"). This behavior on the slope of G is equivalent to requiring $f'''(u) < 0$. For $f(u) = u \ln u$, we in fact have $f'''(u) = -1/u^2 < 0$.

In more picturesque language, the nonlinearity acts to "create" nonzero values for the unmeasured c_i's, so as to suppress the low-intensity ripple and sharpen the point sources.

19.7.1 Is MEM Really Magical?

How unique is the negentropy functional (19.7.3)? Recall that that equation is based on the assumption that luminance elements are a priori distributed over the pixels uniformly. If we instead had some other preferred a priori image in mind, one with pixel intensities m_μ, then it is easy to show that the negentropy becomes

$$H(\mathbf{u}) = \sum_{\mu=0}^{M-1} u_\mu \ln(u_\mu / m_\mu) + \text{constant} \tag{19.7.11}$$

(the constant can then be ignored). All the rest of the discussion then goes through.

More fundamentally, and despite statements by zealots to the contrary [5], there is actually nothing universal about the functional form $f(u) = u \ln u$. In some other physical situations (for example, the entropy of an electromagnetic field in the limit of many photons per mode, as in radio-astronomy) the physical negentropy functional is actually $f(u) = -\ln u$ (see [3] for other examples). In general, the question, "Entropy of what?" is not uniquely answerable in any particular situation. (See reference [7] for an attempt at articulating a more general principle that reduces to one or another entropy functional under appropriate circumstances.)

The four numbered properties summarized above, plus the desirable sign for nonlinearity, $f'''(u) < 0$, are all as true for the function $f(u) = -\ln u$ as for $f(u) = u \ln u$. In fact, these properties are shared by a nonlinear function as simple as $f(u) = -\sqrt{u}$, which has no information-theoretic justification at all (no logarithms!). MEM reconstructions of test images using any of these entropy forms are virtually indistinguishable [3].

By all available evidence, MEM seems to be neither more nor less than one usefully nonlinear version of the general regularization scheme $\mathcal{A} + \lambda \mathcal{B}$ that we have by now considered in many forms. Its peculiarities become strengths when applied to the reconstruction from incomplete Fourier data of images that are expected to be dominated by very bright point sources, but which also contain interesting low-intensity, extended sources. For images of some other character, there is no reason to suppose that MEM methods will generally dominate other regularization schemes, either ones already known or yet to be invented.

19.7.2 Algorithms for MEM

The goal is to find the vector $\hat{\mathbf{u}}$ that minimizes $\mathcal{A} + \lambda \mathcal{B}$ where in the notation of equations (19.5.5), (19.5.6), and (19.7.7),

$$\mathcal{A} = |\mathbf{b} - \mathbf{A} \cdot \mathbf{u}|^2 \qquad \mathcal{B} = \sum_{\mu} f(u_\mu) \tag{19.7.12}$$

Compared with a "general" minimization problem, we have the advantage that we can compute the gradients and the second partial derivative matrices (Hessian matrices) explicitly,

$$\begin{aligned} \nabla \mathcal{A} &= 2(\mathbf{A}^T \cdot \mathbf{A} \cdot \mathbf{u} - \mathbf{A}^T \cdot \mathbf{b}) & \frac{\partial^2 \mathcal{A}}{\partial u_\mu \partial u_\rho} &= [2\mathbf{A}^T \cdot \mathbf{A}]_{\mu\rho} \\ [\nabla \mathcal{B}]_\mu &= f'(u_\mu) & \frac{\partial^2 \mathcal{B}}{\partial u_\mu \partial u_\rho} &= \delta_{\mu\rho} f''(u_\mu) \end{aligned} \tag{19.7.13}$$

It is important to note that while $\mathcal{A}$'s second partial derivative matrix cannot be stored (its size is the square of the number of pixels), it can be applied to any vector by first applying $\mathbf{A}$, then $\mathbf{A}^T$. In the case of reconstruction from incomplete Fourier data, or in the case of convolution with a translation invariant point spread function, these applications will typically involve several FFTs. Likewise, the calculation of the gradient $\nabla \mathcal{A}$ will involve FFTs in the application of $\mathbf{A}$ and $\mathbf{A}^T$.

While some success has been achieved with the classical conjugate gradient method (§10.8), it is often found that the nonlinearity in $f(u) = u \ln u$ causes problems. Attempted steps that give $\mathbf{u}$ with even one negative value must be cut in magnitude, sometimes so severely as to slow the solution to a crawl. The underlying

problem is that the conjugate gradient method develops its information about the inverse of the Hessian matrix a bit at a time, while changing its location in the search space. When a nonlinear function is quite different from a pure quadratic form, the old information becomes obsolete before it gets usefully exploited.

Skilling and collaborators [4,5,8,9] developed a complicated but highly successful scheme, wherein a minimum is repeatedly sought not along a single search direction, but in a small- (typically three-) dimensional subspace, spanned by vectors that are calculated anew at each landing point. The subspace basis vectors are chosen in such a way as to avoid directions leading to negative values. One of the most successful choices is the three-dimensional subspace spanned by the vectors with components given by

$$\begin{aligned} e_\mu^{(1)} &= u_\mu [\nabla \mathcal{A}]_\mu \\ e_\mu^{(2)} &= u_\mu [\nabla \mathcal{B}]_\mu \\ e_\mu^{(3)} &= \frac{u_\mu \sum_\rho (\partial^2 \mathcal{A}/\partial u_\mu \partial u_\rho) u_\rho [\nabla \mathcal{B}]_\rho}{\sqrt{\sum_\rho u_\rho \left([\nabla \mathcal{B}]_\rho\right)^2}} - \frac{u_\mu \sum_\rho (\partial^2 \mathcal{A}/\partial u_\mu \partial u_\rho) u_\rho [\nabla \mathcal{A}]_\rho}{\sqrt{\sum_\rho u_\rho \left([\nabla \mathcal{A}]_\rho\right)^2}} \end{aligned} \tag{19.7.14}$$

(In these equations there is no sum over μ.) The form of the $\mathbf{e}^{(3)}$ has some justification if one views dot products as occurring in a space with the metric $g_{\mu\nu} = \delta_{\mu\nu}/u_\mu$, chosen to make zero values "far away"; see [4].

Within the three-dimensional subspace, the three-component gradient and nine-component Hessian matrix are computed by projection from the large space, and the minimum in the subspace is estimated by (trivially) solving three simultaneous linear equations, as in §10.9, equation (10.9.4). The size of a step $\Delta\mathbf{u}$ is required to be limited by the inequality

$$\sum_\mu (\Delta u_\mu)^2 / u_\mu < (0.1 \text{ to } 0.5) U \tag{19.7.15}$$

Because the gradient directions $\nabla\mathcal{A}$ and $\nabla\mathcal{B}$ are separately available, it is possible to combine the minimum search with a simultaneous adjustment of λ so as finally to satisfy the desired constraint. There are various further tricks employed.

A less general, but in practice often equally satisfactory, approach is due to Cornwell and Evans [10]. Here, noting that $\mathcal{B}$'s Hessian (second partial derivative) matrix is diagonal, one asks whether there is a useful diagonal approximation to $\mathcal{A}$'s Hessian, namely $2\mathbf{A}^T \cdot \mathbf{A}$. If Λ_μ denotes the diagonal components of such an approximation, then a useful step in $\mathbf{u}$ would be

$$\Delta u_\mu = -\frac{1}{\Lambda_\mu + \lambda f''(u_\mu)} (\nabla\mathcal{A} + \lambda \nabla\mathcal{B}) \tag{19.7.16}$$

(again compare equation 10.9.4). Even more extreme, one might seek an approximation with constant diagonal elements, $\Lambda_\mu = \Lambda$, so that

$$\Delta u_\mu = -\frac{1}{\Lambda + \lambda f''(u_\mu)} (\nabla\mathcal{A} + \lambda \nabla\mathcal{B}) \tag{19.7.17}$$

Since $\mathbf{A}^T \cdot \mathbf{A}$ has something of the nature of a doubly convolved point spread function, and since in real cases one often has a point spread function with a sharp

central peak, even the more extreme of these approximations is often fruitful. One starts with a rough estimate of Λ obtained from the $A_{i\mu}$'s, e.g.,

$$\Lambda \sim \left\langle \sum_i [A_{i\mu}]^2 \right\rangle \tag{19.7.18}$$

An accurate value is not important, since in practice Λ is adjusted adaptively: If Λ is too large, then equation (19.7.17)'s steps will be too small (that is, larger steps in the same direction will produce even greater decrease in $\mathcal{A} + \lambda\mathcal{B}$). If Λ is too small, then attempted steps will land in an unfeasible region (negative values of u_μ) or will result in an increased $\mathcal{A} + \lambda\mathcal{B}$. There is an obvious similarity between the adjustment of Λ here and the Levenberg-Marquardt method of §15.5; this should not be too surprising, since MEM is closely akin to the problem of nonlinear least-squares fitting. Reference [10] also discusses how the value of $\Lambda + \lambda f''(u_\mu)$ can be used to adjust the Lagrange multiplier λ so as to converge to the desired value of χ^2.

All practical MEM algorithms are found to require on the order of 30 to 50 iterations to converge. This convergence behavior is not now understood in any fundamental way.

19.7.3 "Bayesian" versus "Historic" Maximum Entropy

Several generalizations of the basic maximum entropy image restoration technique go under the rubric "Bayesian" to distinguish them from the previous "historic" methods. See [11] for details and references. (Our view, of course, is that all the methods are about equally Bayesian, as discussed in §19.4.)

- Better priors: We already noted that the entropy functional (equation 19.7.7) is invariant under scrambling all pixels and has no notion of smoothness. The so-called "intrinsic correlation function" (ICF) model (reference [11], where it is called "New MaxEnt") is similar enough to the entropy functional to allow similar algorithms, but it makes the values of neighboring pixels correlated, enforcing smoothness.
- Better estimation of λ: Above we chose λ to bring χ^2 into its expected narrow statistical range of $N \pm (2N)^{1/2}$. This in effect overestimates χ^2, however, since some effective number γ of parameters are being "fitted" in doing the reconstruction. A Bayesian approach leads to a self-consistent estimate of this γ and an objectively better choice for λ.

CITED REFERENCES AND FURTHER READING:

Gzyl, H. 1995, *The Method of Maximum Entropy* (Singapore: World Scientific).[1]

Wu, N. 1997, *The Maximum Entropy Method* (Berlin: Springer).[2]

Narayan, R., and Nityananda, R. 1986, "Maximum Entropy Image Restoration in Astronomy," *Annual Review of Astronomy and Astrophysics*, vol. 24, pp. 127–170.[3]

Skilling, J., and Bryan, R.K. 1984, "Maximum Entropy Image Reconstruction: General Algorithm," *Monthly Notices of the Royal Astronomical Society*, vol. 211, pp. 111–124.[4]

Burch, S.F., Gull, S.F., and Skilling, J. 1983, "Image Restoration by a Powerful Maximum Entropy Method," *Computer Vision, Graphics and Image Processing*, vol. 23, pp. 113–128.[5]

Skilling, J. 1989, in *Maximum Entropy and Bayesian Methods*, J. Skilling, ed. (Boston: Kluwer).[6]

Frieden, B.R. 1983, "Unified Theory for Estimating Frequency-of-Occurrence Laws and Optical Objects," *Journal of the Optical Society of America*, vol. 73, pp. 927–938.[7]

Skilling, J., and Gull, S.F. 1985, in *Maximum-Entropy and Bayesian Methods in Inverse Problems*, C.R. Smith and W.T. Grandy, Jr., eds. (Dordrecht: Reidel).[8]

Skilling, J. 1986, in *Maximum Entropy and Bayesian Methods in Applied Statistics*, J.H. Justice, ed. (Cambridge, UK: Cambridge University Press).[9]

Cornwell, T.J., and Evans, K.F. 1985, "A Simple Maximum Entropy Deconvolution Algorithm," *Astronomy and Astrophysics*, vol. 143, pp. 77–83.[10]

Gull, S.F. 1989, in *Maximum Entropy and Bayesian Methods*, J. Skilling, ed. (Boston: Kluwer).[11]